

AUTOENCODERS AND GENERATIVE AI

24AM6PCAGA

by,

Dr. Monika Puttaramaiah

Associate Professor

Dept. of MEL, BMSCE

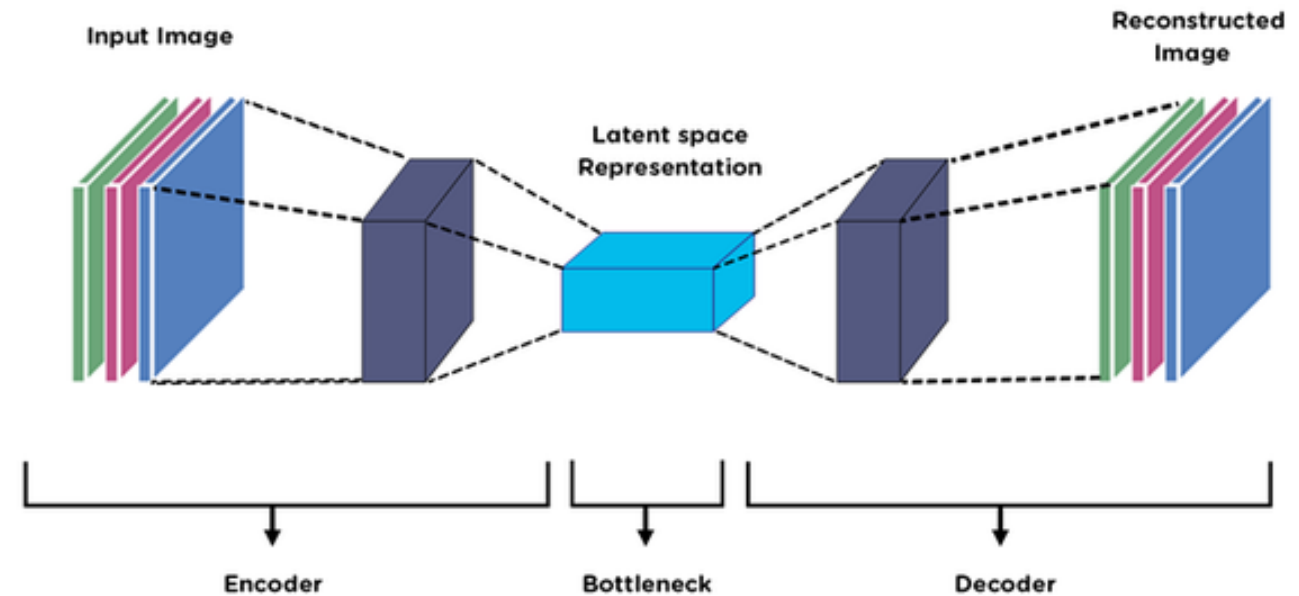
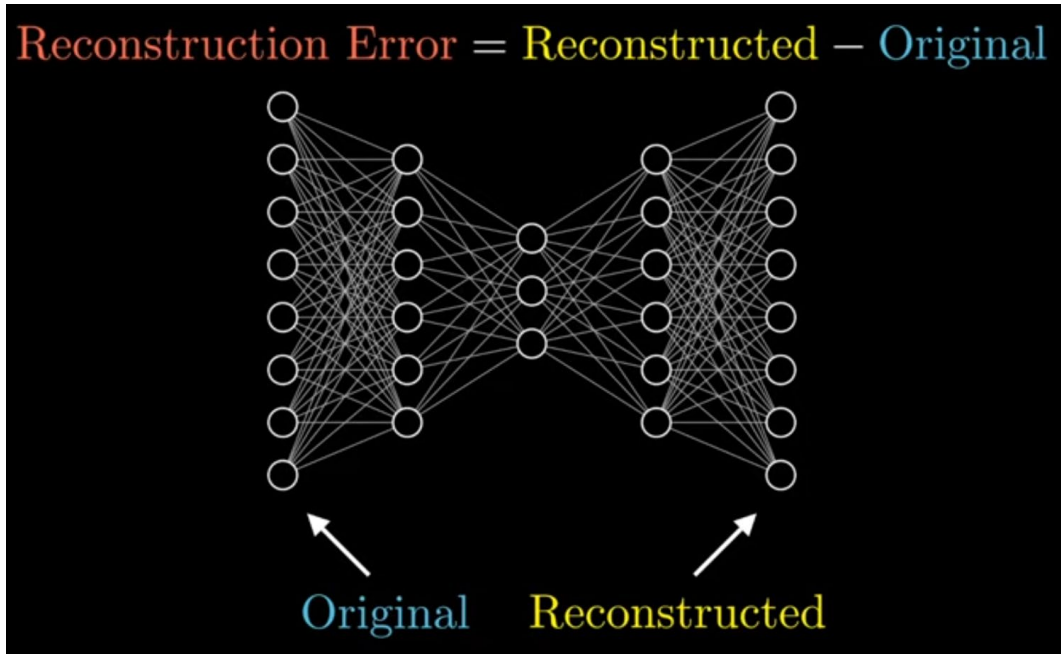
UNIT - 1

→ AUTOENCODERS

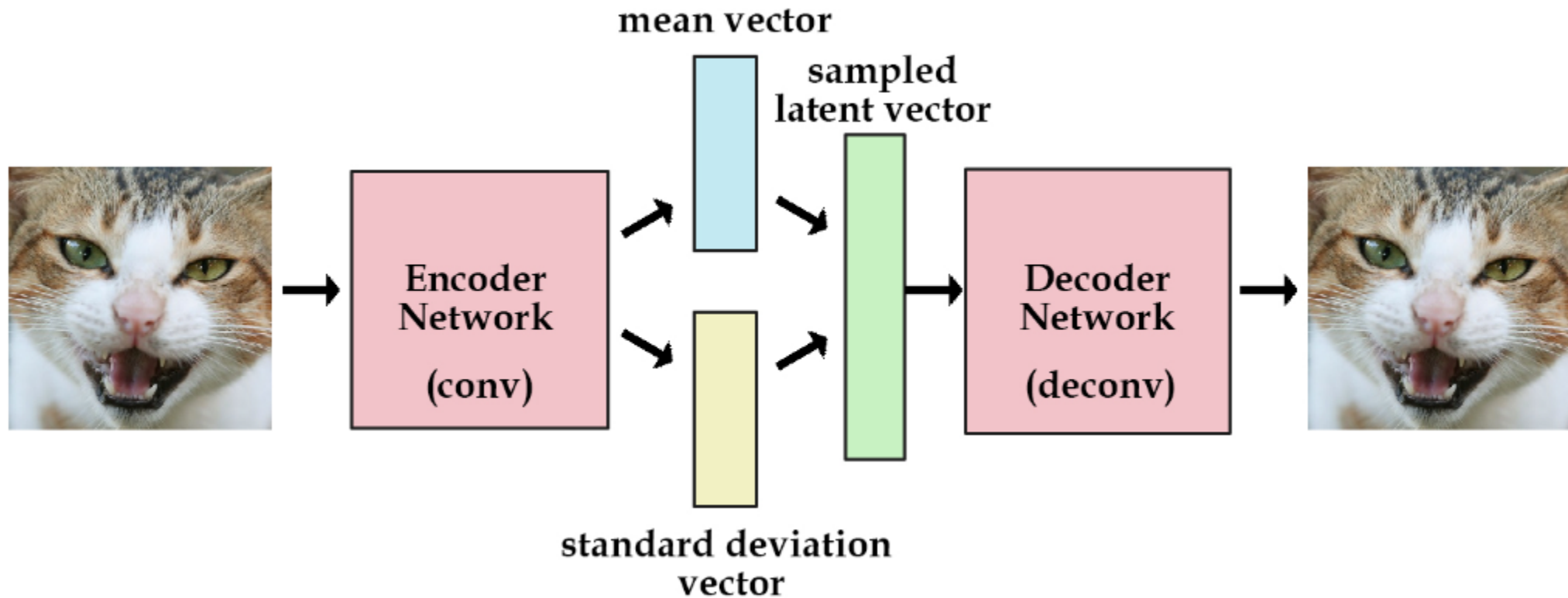
→ VARIATIONAL AUTOENCODERS (VAEs)

Autoencoders

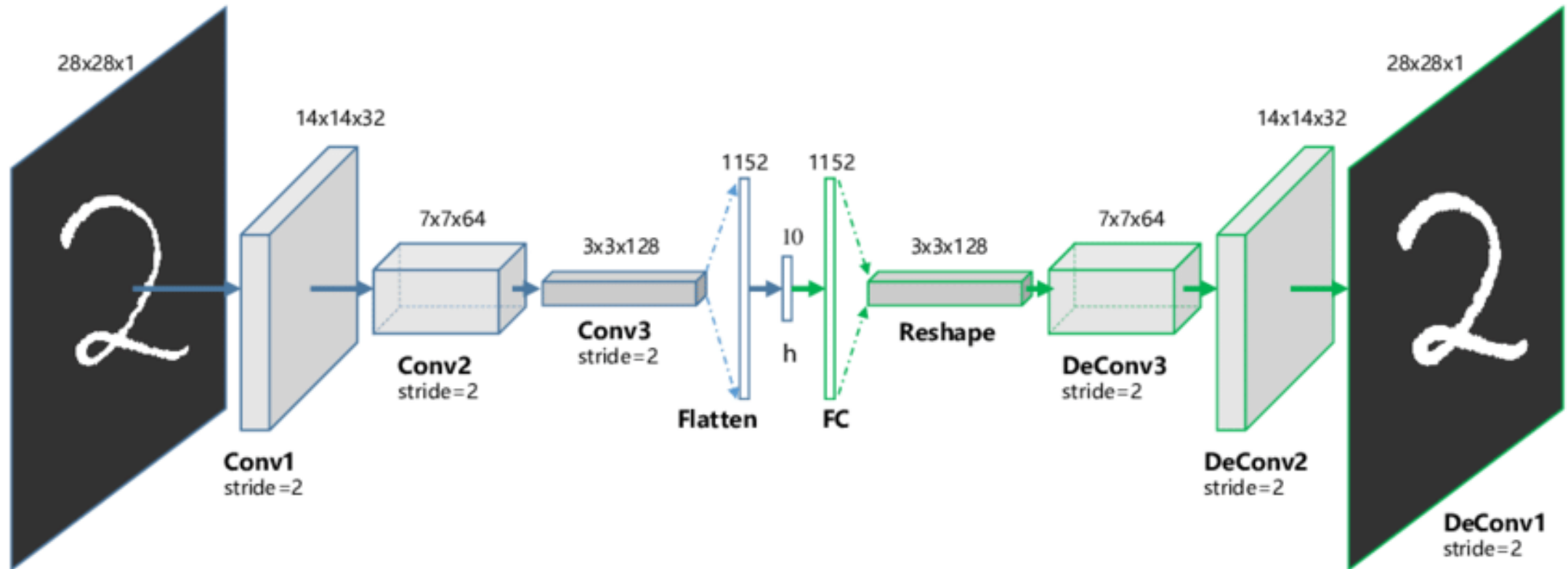
- <https://www.youtube.com/watch?v=hZ4a4NgM3u0>



Autoencoders

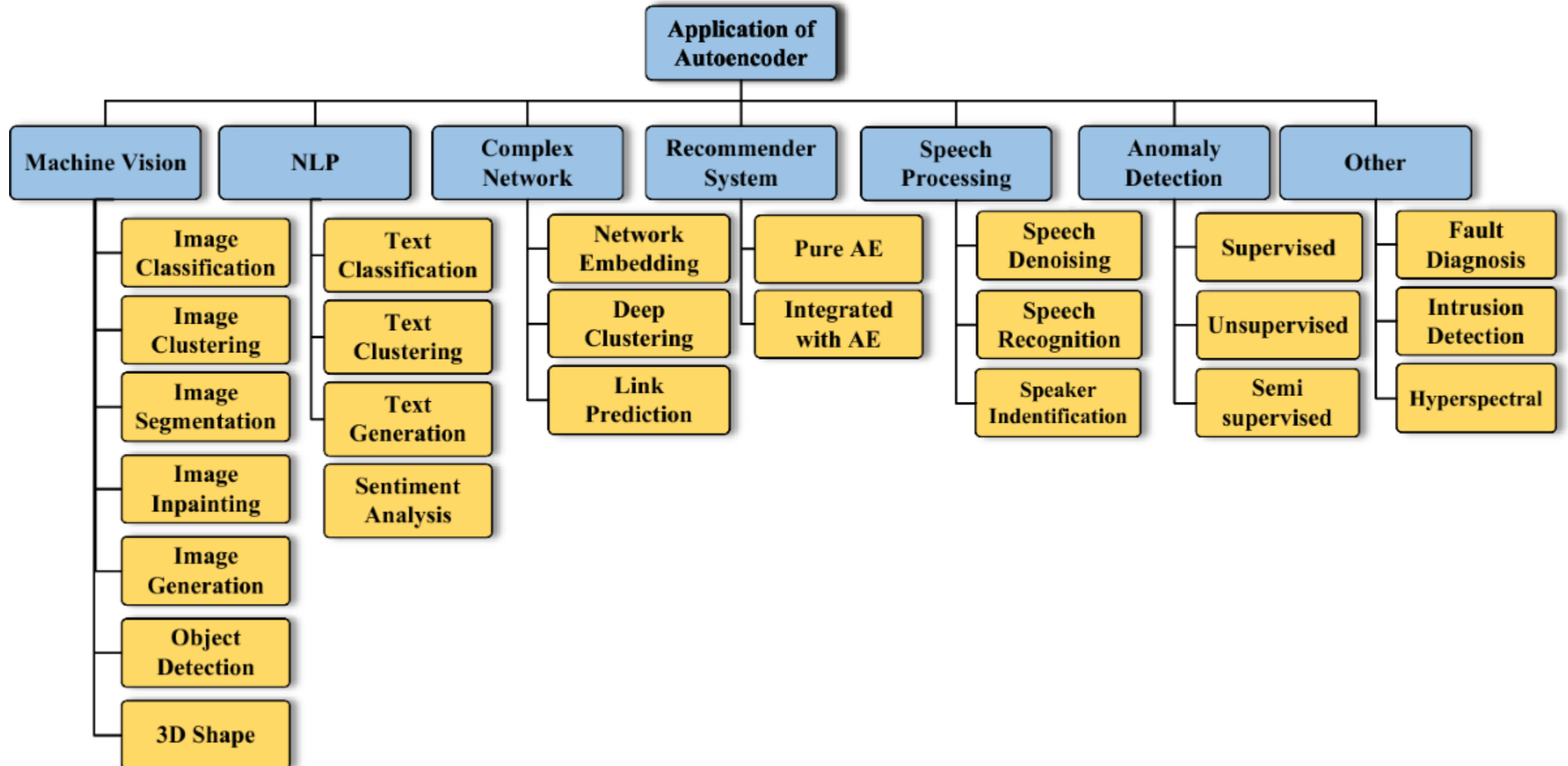


Autoencoders



Dept. of

Autoencoders



Autoencoders

Prominent applications of autoencoders in generative AI

This slide provides information regarding autoencoders as a versatile tool for data preprocessing and has various applications such as image or audio compression, dimensionality reduction, data generation, denoising, recommender system, and anomaly detection.



01

Image and Audio Compression

Autoencoders are suitable to compress high-quality images or audio files while managing critical information. They are trained to restore the original picture or audio from compressed representation

02

Dimensionality Reduction

Models can reduce dimensionality of high-quality datasets. Autoencoders are trained to generate lower-dimensional data representation that captures critical features

03

Data Generation

Deploy autoencoders to create new data similar to training data

04

Denoising

Autoencoders can be utilized to limit noise from data by training model to restore original data from noisy version

05

Recommender System

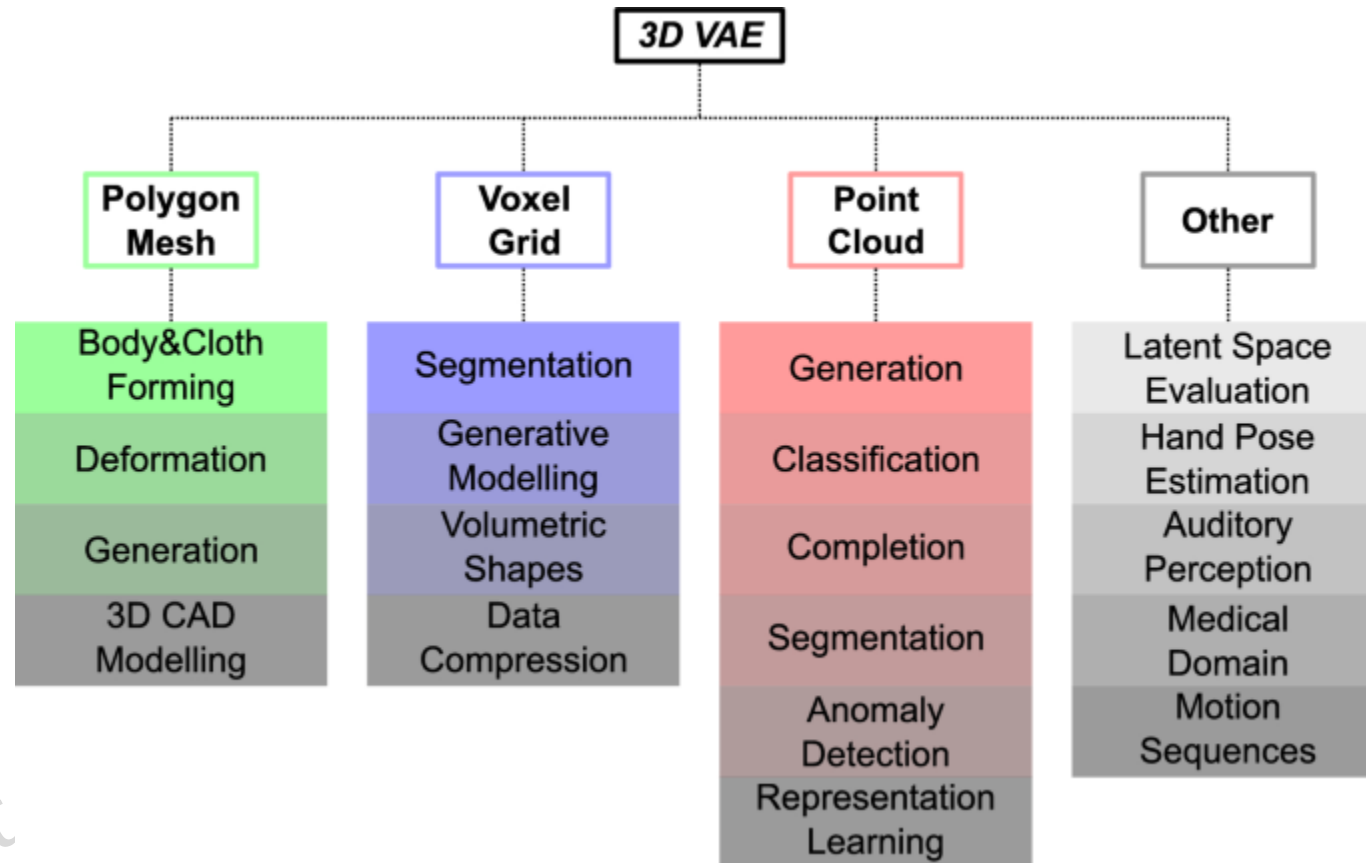
Model can develop customized suggestions as per users' preferences

06

Anomaly Detection

Model can identify anomalies or outliers across dataset through autoencoders by training them over datasets

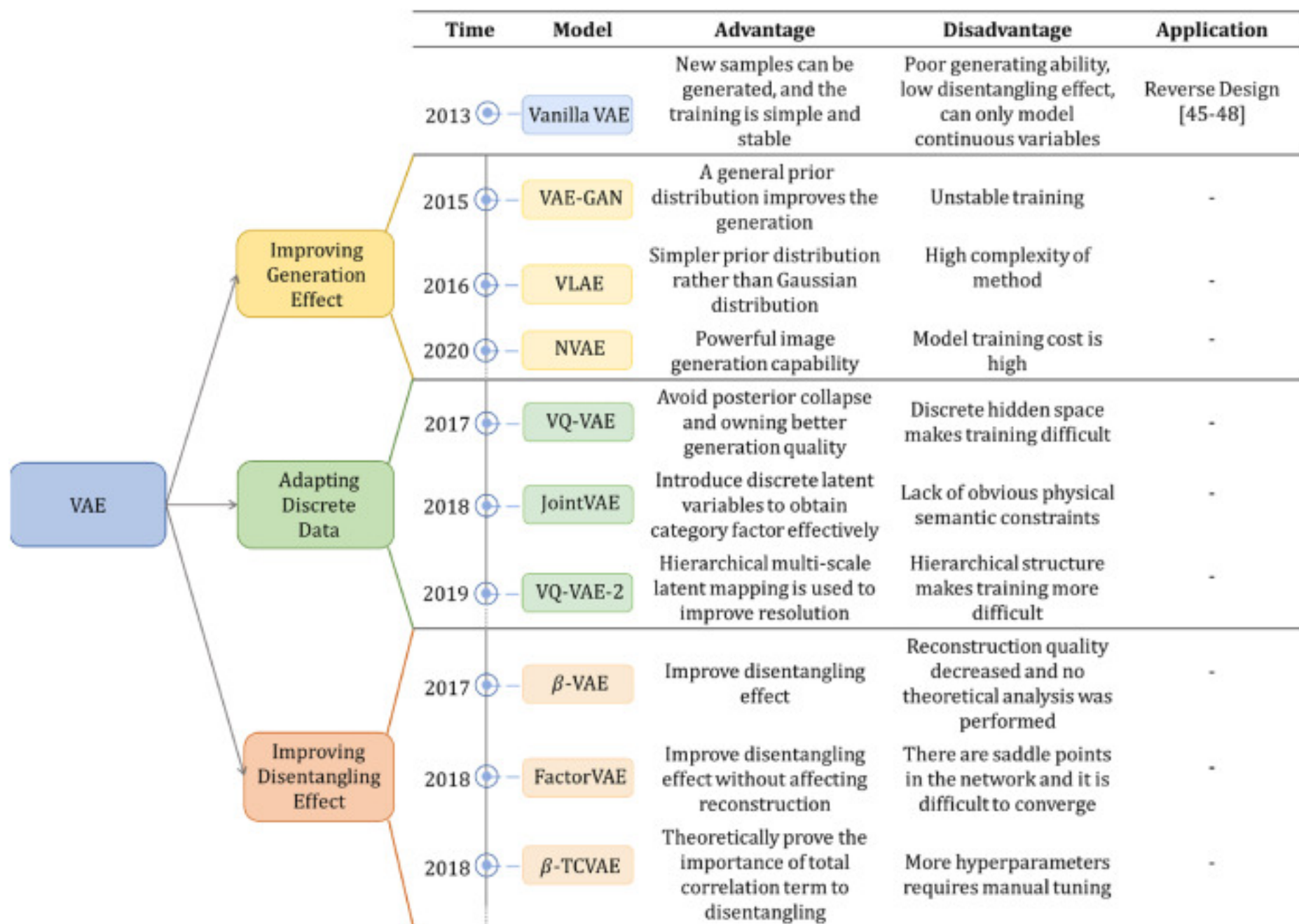
Autoencoders



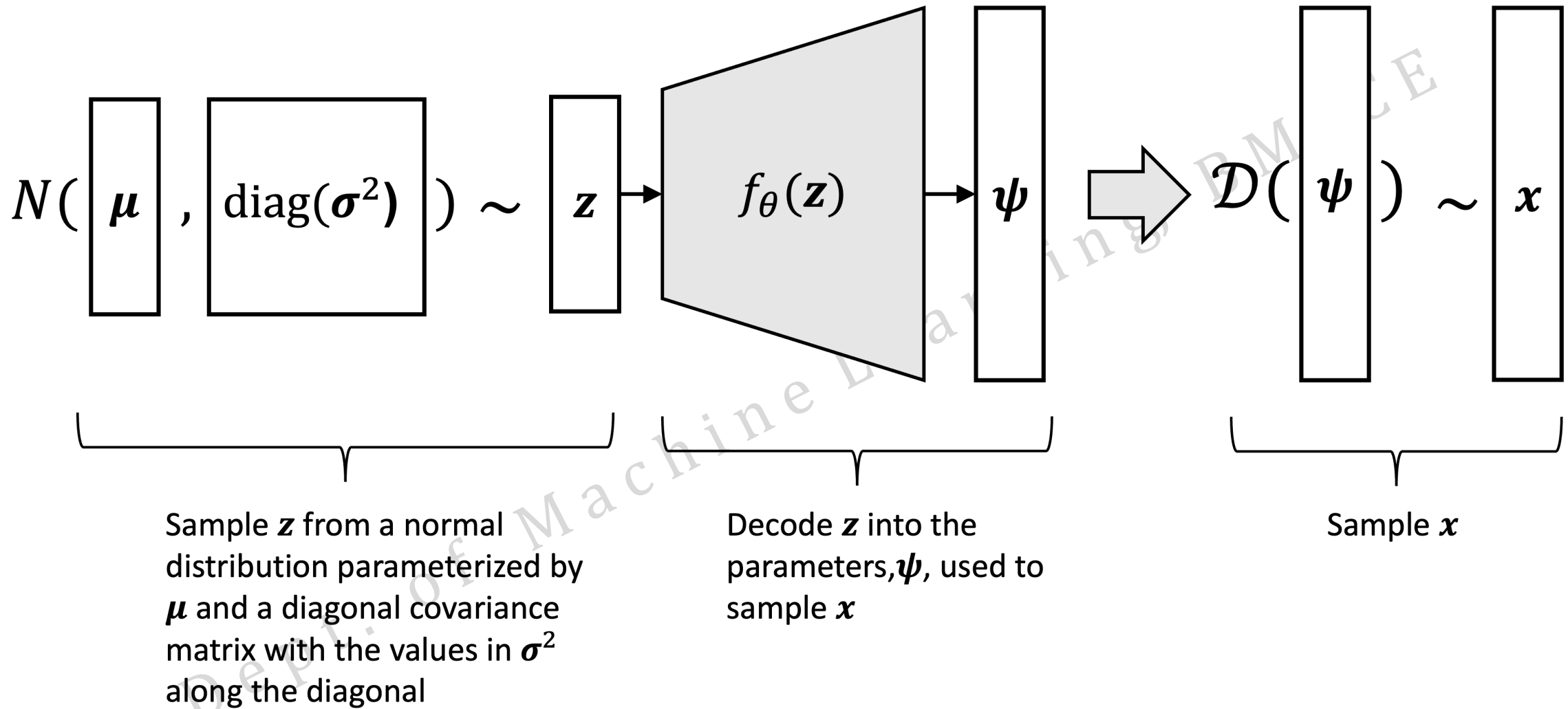
Autoencoders

Comparison	Autoencoder (AE)	Variational Autoencoder (VAE)	Conditional Variational Autoencoder (CVAE)
Data Generation	Not suitable	Suitable	Suitable
Latent Space Regularization	Not present	Present	Present
Latent Space Interpretability	Not present	Present	Present
Handling Missing Data	Not suitable	Suitable	Suitable
Handling high-dimensional data	Not suitable	Suitable	Suitable
Handling complex data	Not suitable	Suitable	Suitable
Handling sequential data	Not suitable	Suitable	Suitable
Anomaly Detection	Suitable	Suitable	Suitable
Handling conditional data	Not suitable	Not suitable	Suitable
Data Reconstruction	Suitable	Suitable	Suitable
Dimensionality Reduction	Suitable	Suitable	Suitable
Handling Multi-modal Data	Not suitable	Suitable	Suitable
Handling Multi-task Data	Not suitable	Not suitable	Suitable

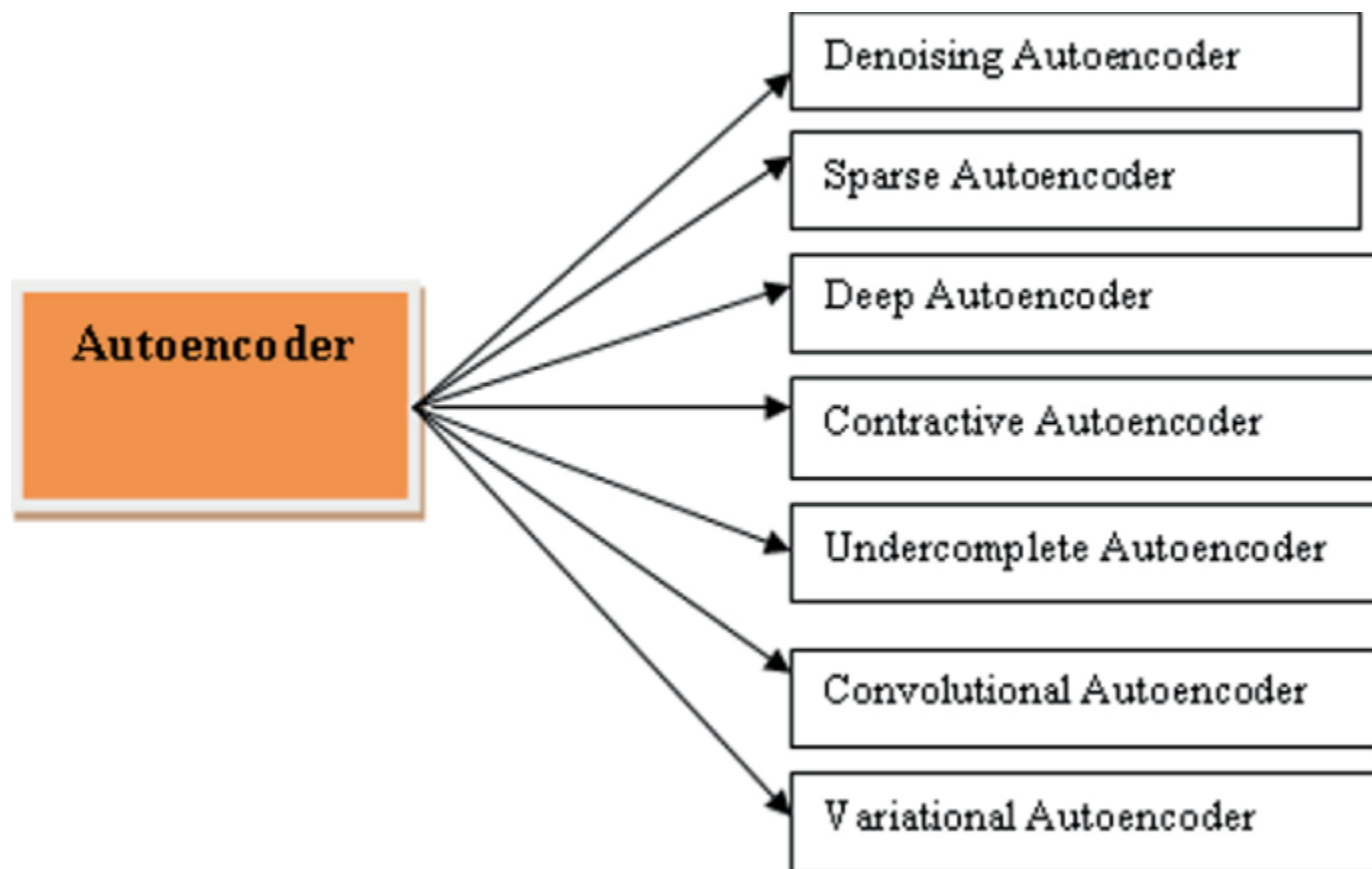
Autoencoders



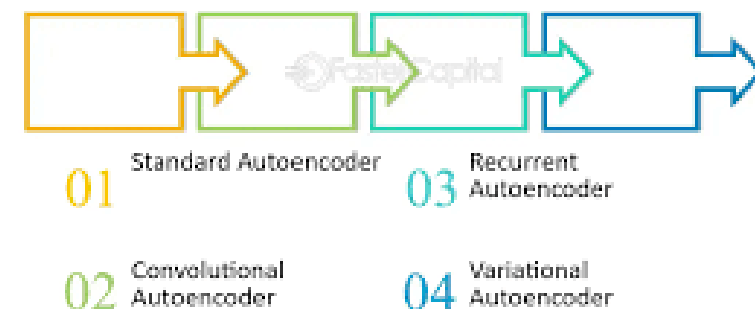
Variational Autoencoders



Autoencoders Types



Types of Autoencoders



Autoencoders

- Lets solve a simple mathematical example

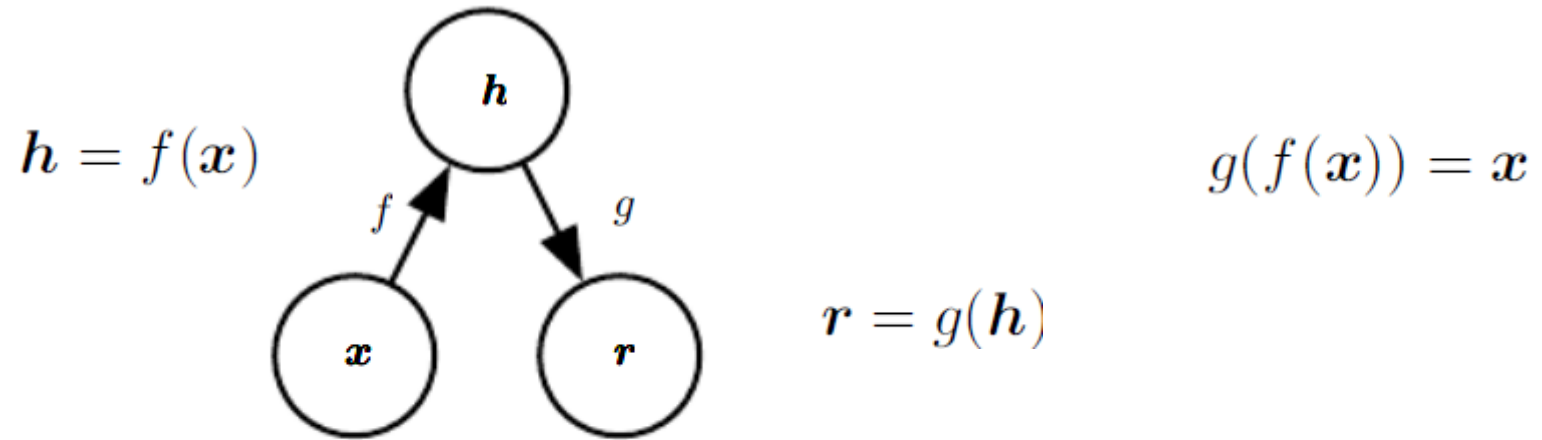


Figure 14.1: The general structure of an autoencoder, mapping an input x to an output (called reconstruction) r through an internal representation or code h . The autoencoder has two components: the encoder f (mapping x to h) and the decoder g (mapping h to r).

Modern autoencoders have generalized the idea of an encoder and a decoder beyond deterministic functions to stochastic mappings $p_{\text{encoder}}(h \mid x)$ and $p_{\text{decoder}}(x \mid h)$.

Autoencoders

- Unlike general feedforward networks, autoencoders may also be trained using **recirculation** (Hinton and McClelland, 1988), a learning algorithm based on comparing the activations of the network on the original input to the activations on the reconstructed input.
- Recirculation is regarded as more biologically plausible than back-propagation, but is rarely used for machine learning applications.

Dept. of Machine Learning, BMSCE

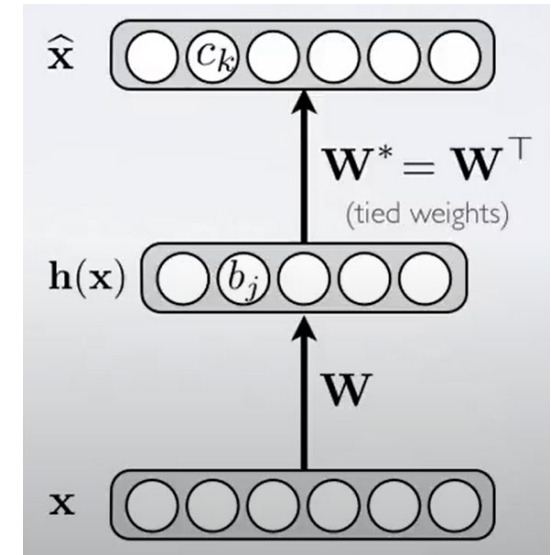
Undercomplete AE

- The dimensionality of the hidden layer (h) is smaller than the dimensionality of the input (x).
- **Objective:** The autoencoder tries to compress the input data into a much smaller space, capturing only the most significant features. This process is similar to **PCA**, which reduces the data's dimensionality while retaining key information.
- **Use Cases:** Dimensionality reduction, feature extraction, and denoising tasks.

The learning process is described simply as minimizing a loss function

$$L(\mathbf{x}, g(f(\mathbf{x}))) \quad (14.1)$$

where L is a loss function penalizing $g(f(\mathbf{x}))$ for being dissimilar from $\mathbf{x}$, such as the mean squared error.



Undercomplete AE

- If decoder is linear and L is MSE, Undercomplete AE learns to span same subspace as PCA (Side effect: copying task)
- Non-linear combination of encoder, decoder functions can learn more powerful nonlinear generalization of PCA. If the autoencoder is of too much capacity, then it does just copying task and doesn't learn anything great from the distribution of data.

Key Features of Undercomplete Autoencoders

- **Dimensionality Reduction** – Since the bottleneck layer has fewer neurons than the input, the model must learn to extract the most essential features.
- **Prevention of Overfitting** – By limiting capacity, the model generalizes better instead of simply copying the input.
- **Feature Learning** – The encoder learns a lower-dimensional representation that captures key patterns in the data.

Undercomplete AE

Mathematical Representation

Given an input x , the encoder maps it to the latent representation h :

$$h = f_{\theta}(x)$$

where f_{θ} is the encoding function (usually a neural network).

The decoder reconstructs the input from h :

$$\hat{x} = g_{\phi}(h)$$

where g_{ϕ} is the decoding function.

The model is trained to minimize the reconstruction error:

$$L(x, \hat{x}) = ||x - \hat{x}||^2$$

where L is a loss function (e.g., Mean Squared Error).

Applications

- ✓ Dimensionality reduction (similar to PCA but nonlinear)
- ✓ Anomaly detection
- ✓ Feature extraction
- ✓ Denoising data

Undercomplete AE

$$x = \begin{bmatrix} 2 \\ 3 \end{bmatrix} \quad W = \begin{bmatrix} 0.6 & 0.8 \end{bmatrix} \text{ (weights of the encoder)} \quad W' = \begin{bmatrix} 0.7 \\ 0.9 \end{bmatrix} \text{ (weights of the decoder)}$$
$$b = 0.5 \text{ (bias)} \quad b' = \begin{bmatrix} 0.2 \\ 0.3 \end{bmatrix} \text{ (biases)}$$
$$h = f_{\theta}(x) = Wx + b \quad \hat{x} = g_{\phi}(h) = W'h + b'$$

Compute Reconstruction Error!

Overcomplete AE

Overcomplete Autoencoders: Hidden dimension is greater than the input → Just does a copy task

- An overcomplete autoencoder is an autoencoder where the dimensionality of the latent space (bottleneck layer) is larger than the input data dimension.
- Unlike an undercomplete autoencoder, which forces a compressed representation, an overcomplete autoencoder has more capacity in the latent space, which can sometimes lead to identity mapping (trivial solutions) if not properly regularized.

Key Features (Mathematical representation – same functions as in Undercomplete AE):

- **Higher Latent Dimension** – The bottleneck layer has more neurons than the input layer.
- **Risk of Learning Identity Function** – Without constraints (e.g., sparsity, regularization), the autoencoder might simply copy the input to output.
- **Used for Feature Learning** – If regularized properly (e.g., sparse autoencoders), it can learn useful high-dimensional feature representations.

Overcomplete AE

$$x = 5 \quad W = \begin{bmatrix} 0.6 \\ 0.9 \end{bmatrix} \text{ (weights)}$$

$$b = \begin{bmatrix} 0.2 \\ 0.1 \end{bmatrix} \text{ (biases)}$$

$$W' = \begin{bmatrix} 0.5 & 0.3 \end{bmatrix} \text{ (weights)}$$

$$b' = 0.4 \text{ (bias)}$$

$$h = f_{\theta}(x) = Wx + b$$

$$\hat{x} = g_{\phi}(h) = W'h + b'$$

Compute Reconstruction Error!

Regularized AE

- A regularized autoencoder is an autoencoder that applies constraints to the learning process to prevent trivial solutions (such as learning the identity function) and encourage meaningful feature extraction.
- These constraints help the model generalize better and learn useful representations instead of just memorizing the input.
- Ideally, one could train any architecture of autoencoder successfully, choosing the code dimension and the capacity of the encoder and decoder based on the complexity of distribution to be modeled.
- Other properties include sparsity of the representation, smallness of the derivative of the representation, and robustness to noise or to missing inputs. A regularized autoencoder can be nonlinear and overcomplete but still learn something useful about the data distribution even if the model capacity is great enough to learn a trivial identity function.

Regularized AE

- Descendants of the Helmholtz machine (Hinton et al., 1995b), such as the variational autoencoder (section) and the generative stochastic networks...
- These models naturally learn high-capacity, overcomplete encodings of the input and do not require regularization for these encodings to be useful.
- Their encodings are naturally useful because the models were trained to approximately maximize the probability of the training data rather than to copy the input to the output.

Dept. of Machine Learning BMSCE

Error Computation

Mean Squared Error (MSE)

- **Description:** The Mean Squared Error (MSE) loss function is commonly used for **real-valued inputs**. It calculates the squared difference between the input data and the reconstructed data, averaging over all data points.

Formula:

$$MSE = \frac{1}{N} \sum_{i=1}^N (x_i - \hat{x}_i)^2$$

where x_i is the original input and $\hat{x}_i$ is the reconstructed input.

Der

Error Computation

Cross-Entropy Loss

- **Description:** For binary inputs, the cross-entropy loss is often preferred. It compares the predicted probabilities from the sigmoid function to the actual binary values (0 or 1). This is more effective than MSE in cases where the data consists of binary values.

Formula:

$$L_{\text{CE}} = -\frac{1}{N} \sum_{i=1}^N [x_i \log(\hat{x}_i) + (1 - x_i) \log(1 - \hat{x}_i)]$$

where x_i is the original binary input and $\hat{x}_i$ is the predicted probability.

Error Computation

Reconstruction losses

$$\mathcal{L} = \frac{1}{m} \sum_{j=1}^m \ell(\mathbf{x}^{(j)}, \hat{\mathbf{x}}^{(j)})$$

binary input

$$\ell(\mathbf{x}, \hat{\mathbf{x}}) = - \sum_{i=1}^n [x_i \log(\hat{x}_i) + (1 - x_i) \log(1 - \hat{x}_i)]$$

real valued input

$$\ell(\mathbf{x}, \hat{\mathbf{x}}) = \frac{1}{2} \|\mathbf{x} - \hat{\mathbf{x}}\|^2$$

Regularized AE - Types

- Sparse Autoencoders (SAE) – Encourage sparsity in the latent representation.
- Denoising Autoencoders (DAE) – Train on corrupted inputs to learn robust features.
- Contractive Autoencoders (CAE) – Penalize sensitivity to small input changes.
- Variational Autoencoders (VAE) – Force the latent space to follow a probabilistic distribution.

Dept. of Machine Learning BMSCE

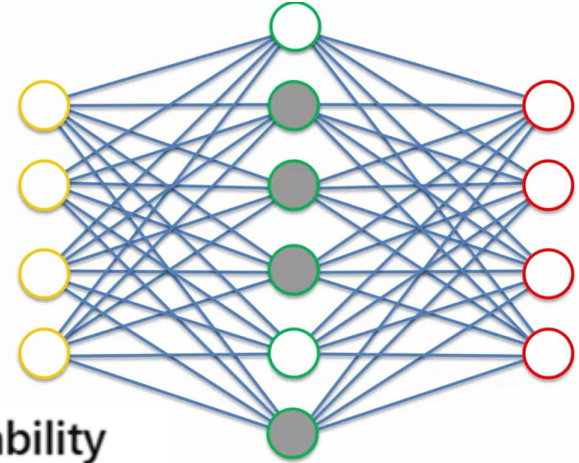
Regularized AE – Sparse AE

- Regularization: Uses L1 regularization (sparsity constraint) on the latent space, ensuring that only a few neurons activate at a time. Loss Function:

Probabilistic interpretation of Autoencoders! – it is standard in Latent Variable models

$$L(x, \hat{x}) = ||x - \hat{x}||^2 + \lambda \sum |h|$$

where h is the latent representation and λ controls the sparsity penalty.



Application: Feature learning, anomaly detection.

$$\log p_{\text{model}}(x) = \log \sum_h p_{\text{model}}(h, x).$$

(14.3) **Factorization of Joint Probability**

$$\log p_{\text{model}}(h, x) = \log p_{\text{model}}(h) + \log p_{\text{model}}(x | h). \quad (14.4)$$

Interpretation

- $p_{\text{model}}(h)$: prior on latent variables
- $p_{\text{model}}(x | h)$: likelihood / decoder model

Sparsity is enforced by choosing a sparsity-promoting prior on h !

The $\log p_{\text{model}}(h)$ term can be sparsity-inducing. For example, the Laplace prior,

$$p_{\text{model}}(h_i) = \frac{\lambda}{2} e^{-\lambda |h_i|}, \quad (14.5)$$

corresponds to an absolute value sparsity penalty. Expressing the log-prior as an absolute value penalty, we obtain

$$\Omega(h) = \lambda \sum_i |h_i| \quad (14.6)$$

$$-\log p_{\text{model}}(h) = \sum_i \left(\lambda |h_i| - \log \frac{\lambda}{2} \right) = \Omega(h) + \text{const} \quad (14.7)$$

Terminology

- This is a **Laplace** (double exponential) distribution
- Strongly peaked at **zero**
- Encourages many values of $h_i = 0$

✓ This is the **probabilistic equivalent of L1 regularization**

Regularized AE – Sparse AE

Why L1 → Sparsity (Intuition)

- L1 penalty has **sharp corner at zero**
- Gradient pushes small activations **exactly to zero**
- Results in:
 - Feature selection
 - Disentangled representations
 - Better interpretability

Applications (Why Sparse AE?)

- **Feature learning**
- **Anomaly detection**
- Denoising
- Dimensionality reduction with interpretability

Regularized AE – Denoising AE

- Regularization: Introduces random noise to the input before encoding, forcing the network to reconstruct the original clean input.

Loss Function:

$$L(x, \hat{x}) = ||x - \hat{x}||^2$$

but with **corrupted** x' as input.

Application: Robust feature extraction, denoising images.

A **denoising autoencoder** or DAE instead minimizes

$$L(x, g(f(\tilde{x}))),$$

Regularized AE – Contractive AE

- Regularization: Adds a Jacobian penalty to the loss function, discouraging large variations in the latent space.

Loss Function:

$$L(x, \hat{x}) = ||x - \hat{x}||^2 + \lambda ||\nabla h||^2$$

where ∇h is the Jacobian of the encoder output.

Application: Extracting stable representations, preventing overfitting.

Regularizing by Penalizing Derivatives

The loss function consists of two terms:

$$L(x, \hat{x}) = ||x - \hat{x}||^2 + \lambda ||\nabla h||^2$$

where:

- $||x - \hat{x}||^2$ is the **reconstruction loss** (mean squared error).
- $||\nabla h||^2$ is the **contractive loss**, penalizing large gradients of h w.r.t x .
- λ is a regularization parameter.

Another strategy for regularizing an autoencoder is to use a penalty Ω as in sparse autoencoders,

$$L(x, g(f(x))) + \Omega(h, x), \quad (14.10)$$

but with a different form of Ω :

$$\Omega(h, x) = \lambda \sum_i ||\nabla_x h_i||^2. \quad (14.11)$$

Regularized AE – Variational AE

- Regularization: Forces the latent space to follow a Gaussian distribution (as latent space is probabilistic) using Kullback-Leibler (KL) divergence.

Loss Function:

Application: Generative models, data augmentation.

$$L(x, \hat{x}) = ||x - \hat{x}||^2 + KL(q(h|x)||p(h))$$

$q(h|x)$: Encoder Distribution

- Output of the encoder
- A **Gaussian distribution**, not a single point

$$q(h|x) = \mathcal{N}(\mu(x), \sigma^2(x))$$

The encoder learns:

- Mean μ
- Variance σ^2

$p(h)$: Prior Distribution

- Chosen beforehand
- Usually a **standard normal distribution**

$$p(h) = \mathcal{N}(0, I)$$

Why Gaussian Regularization?

Because it allows:

- Sampling**
 - We can sample $h \sim \mathcal{N}(0, I)$
- Smooth interpolation**
 - Nearby latent points produce similar outputs
- Generative capability**
 - New data can be generated easily

KL divergence: Penalizes the encoder if $q(h|x)$ deviates from $\mathcal{N}(0, I)$

Final Interpretation of the Loss

$$\underbrace{||x - \hat{x}||^2}_{\text{Reconstruction Accuracy}} + \underbrace{KL(q(h|x)||p(h))}_{\text{Latent Space Regularization}}$$

- First term → **How well we reconstruct**
- Second term → **How well the latent space follows a Gaussian**

The model balances **fidelity vs structure**.

Representational Power, Layer Size and Depth

Traditional vs. Deep Architectures

- Traditional autoencoders use a single-layer encoder and decoder.
- Deep autoencoders incorporate multiple hidden layers.

Advantages of Depth

- **Universal Approximation:** Deeper networks can approximate any function with sufficient units.
- **Enhanced Flexibility:** Deeper mappings allow for enforcing constraints (e.g., sparsity).
- **Efficiency Gains:** Depth can exponentially reduce computational costs and required training data.

Empirical Benefits

- Deep autoencoders achieve superior compression compared to shallow/linear versions (Hinton & Salakhutdinov, 2006).

Training Strategy

- Often trained by greedily pretraining a stack of shallow autoencoders before fine-tuning the deep model.

Stochastic Encoders and Decoders

In the case of an autoencoder, $\mathbf{x}$ is now the target as well as the input. However, we can still apply the same machinery as before. Given a hidden code $\mathbf{h}$, we may think of the decoder as providing a conditional distribution $p_{\text{decoder}}(\mathbf{x} \mid \mathbf{h})$. We may then train the autoencoder by minimizing $-\log p_{\text{decoder}}(\mathbf{x} \mid \mathbf{h})$.

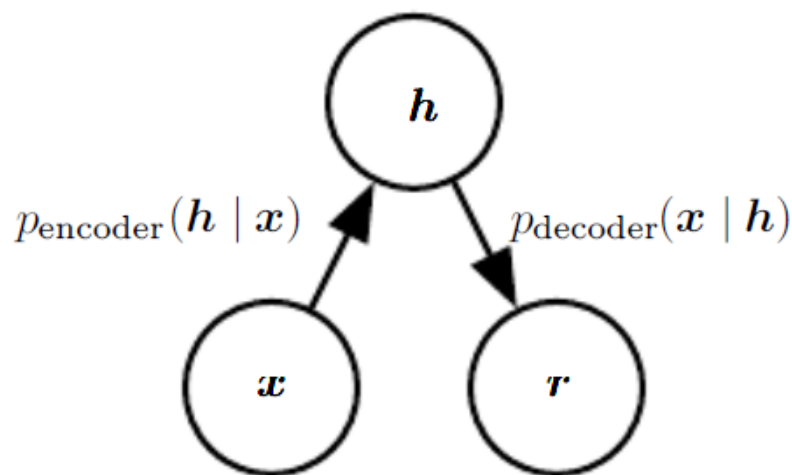


Figure 14.2: The structure of a stochastic autoencoder, in which both the encoder and the decoder are not simple functions but instead involve some noise injection, meaning that their output can be seen as sampled from a distribution, $p_{\text{encoder}}(\mathbf{h} \mid \mathbf{x})$ for the encoder and $p_{\text{decoder}}(\mathbf{x} \mid \mathbf{h})$ for the decoder.

Stochastic Encoders and Decoders

Any latent variable model $p_{\text{model}}(\mathbf{h}, \mathbf{x})$ defines a stochastic encoder

$$p_{\text{encoder}}(\mathbf{h} \mid \mathbf{x}) = p_{\text{model}}(\mathbf{h} \mid \mathbf{x})$$

and a stochastic decoder

$$p_{\text{decoder}}(\mathbf{x} \mid \mathbf{h}) = p_{\text{model}}(\mathbf{x} \mid \mathbf{h}).$$

Dept. of M a c

Stochastic Encoders and Decoders

- **Stochastic Encoder:**

- Maps an input x to a latent probability distribution $q(z|x)$.
- Typically modeled as a Gaussian distribution, parameterized by a mean $\mu(x)$ and a standard deviation $\sigma(x)$.
- Uses the reparameterization trick: a latent variable is sampled as

$$z = \mu(x) + \sigma(x) \cdot \epsilon$$

where $\epsilon \sim \mathcal{N}(0, 1)$.

- **Stochastic Decoder:**

- Takes a latent sample z and maps it to a reconstruction $\hat{x}$ through a conditional probability distribution $p(x|z)$.
- The randomness in the decoder accounts for uncertainty in reconstructing x from z .



Stochastic Encoders and Decoders

Stochasticity:

- The randomness introduced in sampling ϵ means that even with the same x , different samples of z (and thus different reconstructions $x^\wedge$) can occur, helping the model learn a smooth latent space.

Reparameterization Trick:

- This trick allows gradients to flow through the stochastic sampling process during training, making the model end-to-end trainable.

Applications:

- Such stochastic encoders and decoders are central to Variational Autoencoders, used in tasks like generative modeling, anomaly detection, and unsupervised learning.

Stochastic Encoders and Decoders

1. Input and Encoder Parameters:

- Suppose we have an input x (e.g., a data point value).
- The encoder processes x and outputs:
 - Mean: $\mu = 5$
 - Standard Deviation: $\sigma = 2$

2. Sampling the Latent Variable:

- Using the reparameterization trick, sample ϵ from a standard normal distribution.
- Assume we obtain $\epsilon = 0.5$.
- Compute the latent variable:

$$z = \mu + \sigma \cdot \epsilon = 5 + 2 \times 0.5 = 6$$

3. Decoding to Reconstruct the Input:

- The decoder takes z and reconstructs $\hat{x}$.
- Suppose the decoder is a simple linear function:

$$\hat{x} = 2 \times z$$

- Then:

$$\hat{x} = 2 \times 6 = 12$$

Denoising Autoencoders

- The **denoising autoencoder** (DAE) is an autoencoder that receives a corrupted data point as input and is trained to predict the original, uncorrupted data point as its output.

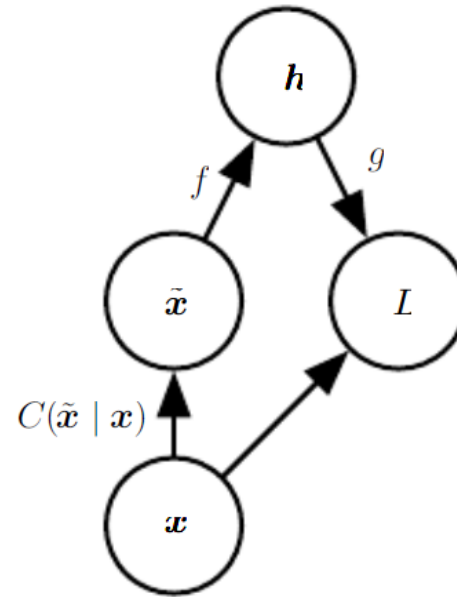


Figure 14.3: The computational graph of the cost function for a denoising autoencoder, which is trained to reconstruct the clean data point $\mathbf{x}$ from its corrupted version $\tilde{\mathbf{x}}$. This is accomplished by minimizing the loss $L = -\log p_{\text{decoder}}(\mathbf{x} | \mathbf{h} = f(\tilde{\mathbf{x}}))$, where $\tilde{\mathbf{x}}$ is a corrupted version of the data example $\mathbf{x}$, obtained through a given corruption process $C(\tilde{\mathbf{x}} | \mathbf{x})$. Typically the distribution p_{decoder} is a factorial distribution whose mean parameters are emitted by a feedforward network g .

Denoising Autoencoders

corruption process $C(\tilde{\mathbf{x}} \mid \mathbf{x})$ which represents a conditional distribution over

corrupted samples $\tilde{\mathbf{x}}$, given a data sample $\mathbf{x}$. The autoencoder then learns a **reconstruction distribution** $p_{\text{reconstruct}}(\mathbf{x} \mid \tilde{\mathbf{x}})$ estimated from training pairs $(\mathbf{x}, \tilde{\mathbf{x}})$, as follows:

1. Sample a training example $\mathbf{x}$ from the training data.
2. Sample a corrupted version $\tilde{\mathbf{x}}$ from $C(\tilde{\mathbf{x}} \mid \mathbf{x} = \mathbf{x})$.
3. Use $(\mathbf{x}, \tilde{\mathbf{x}})$ as a training example for estimating the autoencoder reconstruction distribution $p_{\text{reconstruct}}(\mathbf{x} \mid \tilde{\mathbf{x}}) = p_{\text{decoder}}(\mathbf{x} \mid \mathbf{h})$ with $\mathbf{h}$ the output of encoder $f(\tilde{\mathbf{x}})$ and p_{decoder} typically defined by a decoder $g(\mathbf{h})$.

Denoising Autoencoders

Typically we can simply perform gradient-based approximate minimization (such as minibatch gradient descent) on the negative log-likelihood $-\log p_{\text{decoder}}(\mathbf{x} \mid \mathbf{h})$. So long as the encoder is deterministic, the denoising autoencoder is a feedforward network and may be trained with exactly the same techniques as any other feedforward network.

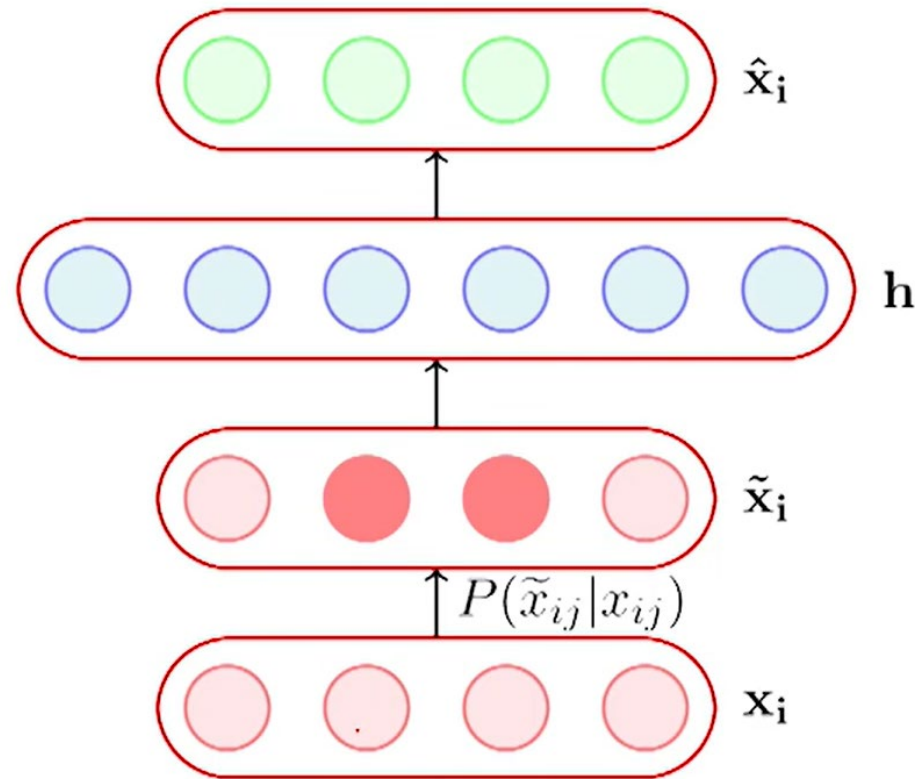
We can therefore view the DAE as performing stochastic gradient descent on the following expectation:

$$-\mathbb{E}_{\mathbf{x} \sim \hat{p}_{\text{data}}(\mathbf{x})} \mathbb{E}_{\tilde{\mathbf{x}} \sim C(\tilde{\mathbf{x}} \mid \mathbf{x})} \log p_{\text{decoder}}(\mathbf{x} \mid \mathbf{h} = f(\tilde{\mathbf{x}})) \quad (14.14)$$

where $\hat{p}_{\text{data}}(\mathbf{x})$ is the training distribution.

Dept.

Denoising Autoencoders



This helps because the objective is still to reconstruct the original (un-corrupted) $\mathbf{x}_i$

$$\arg \min_{\theta} \frac{1}{m} \sum_{i=1}^m \sum_{j=1}^n (\hat{x}_{ij} - x_{ij})^2$$

Denoising Autoencoders – Estimating the Score

$$\nabla_{\mathbf{x}} \log p(\mathbf{x})$$

- This vector field estimates the score $\nabla_{\mathbf{x}} \log p_{\text{data}}(\mathbf{x})$ up to a multiplicative factor that is the average root mean square reconstruction error.
- ✓ A very important property of DAEs is that their training criterion (with conditionally Gaussian $p(\mathbf{x}|\mathbf{h})$) makes the autoencoder learn a vector field $(g(f(\mathbf{x})) - \mathbf{x})$ that estimates the score of the data distribution.

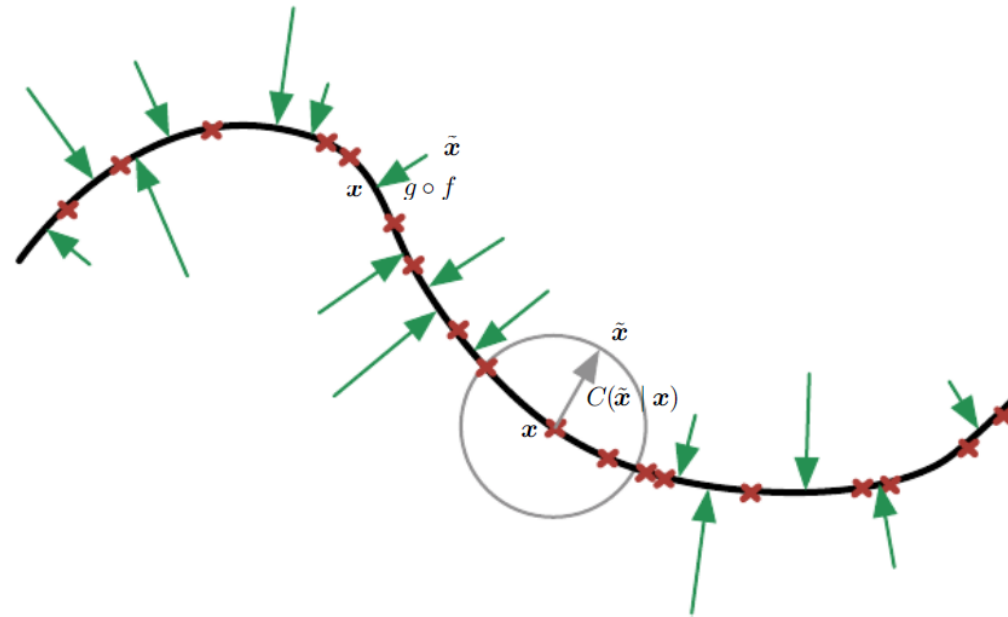


Figure 14.4: A denoising autoencoder is trained to map a corrupted data point $\tilde{\mathbf{x}}$ back to the original data point $\mathbf{x}$. We illustrate training examples $\mathbf{x}$ as red crosses lying near a low-dimensional manifold illustrated with the bold black line. We illustrate the corruption process $C(\tilde{\mathbf{x}} | \mathbf{x})$ with a gray circle of equiprobable corruptions. A gray arrow demonstrates how one training example is transformed into one sample from this corruption process. When the denoising autoencoder is trained to minimize the average of squared errors $\|g(f(\tilde{\mathbf{x}})) - \mathbf{x}\|^2$, the reconstruction $g(f(\tilde{\mathbf{x}}))$ estimates $\mathbb{E}_{\mathbf{x}, \tilde{\mathbf{x}} \sim p_{\text{data}}(\mathbf{x}) C(\tilde{\mathbf{x}} | \mathbf{x})}[\mathbf{x} | \tilde{\mathbf{x}}]$. The vector $g(f(\tilde{\mathbf{x}})) - \tilde{\mathbf{x}}$ points approximately towards the nearest point on the manifold, since $g(f(\tilde{\mathbf{x}}))$ estimates the center of mass of the clean points $\mathbf{x}$ which could have given rise to $\tilde{\mathbf{x}}$. The autoencoder thus learns a vector field $g(f(\mathbf{x})) - \mathbf{x}$ indicated by the green arrows. This vector field estimates the score $\nabla_{\mathbf{x}} \log p_{\text{data}}(\mathbf{x})$ up to a multiplicative factor that is the average root mean square reconstruction error.

Denoising Autoencoders – Estimating the Score

RBM as a Probabilistic Model:

- Provides an explicit probability model $p(\mathbf{x}; \theta)$.
- Can be trained using **denoising score matching** (Kingma & LeCun, 2010).

Connection to Autoencoders:

- RBM learning with denoising score matching is equivalent to **denoising autoencoder training**.
- Regularized score matching may produce a **blurred distribution**, but **consistency is restored** when the noise level **approaches zero** with more data.

Score Matching and Cost Function:

- Score matching applied to RBMs results in a cost function similar to **reconstruction error** with **contractive penalty** (Swersky et al., 2011).
- Autoencoder gradients approximate **contrastive divergence training** of RBMs (Bengio & Delalleau, 2009).

Denoising Autoencoders – Estimating the Score

Denoising Criterion for Continuous Data:

- Gaussian corruption and reconstruction distribution estimate the **score function** (Alain & Bengio, 2013).
- Generic encoder-decoder architectures can estimate the score by training with **squared error loss**.

Constraints on Reconstruction and Score Function:

- In general, **reconstruction minus input** ($g(f(x)) - x$) does **not always correspond** to the gradient of any function.
- Certain **specialized parameterizations** (Vincent, 2011) allow this relationship to hold.
- Kamyshanska & Memisevic (2015) **generalized** these results for a **family of shallow autoencoders** where the score function holds for all members.

Denoising Autoencoders – Historic Perspective

Early Work on Denoising with MLPs:

- LeCun (1987) and Gallinari et al. (1987) pioneered the use of **MLPs for denoising**.
- Behnke (2001) applied **recurrent networks** for image denoising.

Denoising Autoencoders (DAEs):

- DAEs are **MLPs trained to denoise**, but their goal is also to learn **good internal representations** (Vincent et al., 2008, 2010).
- These learned representations can be used to **pretrain deeper networks** (both supervised & unsupervised).

Relation to Other Autoencoders:

- Similar to **sparse autoencoders, sparse coding, and contractive autoencoders**.
- Designed to **learn high-capacity encoders** while preventing them from learning **trivial identity functions**.

Denoising Autoencoders – Historic Perspective

Earlier Related Work (Inayoshi & Kurita, 2005):

- Combined **reconstruction error** with a **supervised objective**.
- Used **noise injection** in the hidden layer of an MLP to improve **generalization**.
- Based on a **linear encoder**, so it lacked the ability to learn **complex functions** like modern DAEs.

Dept. of Machine Learning, BMSCE

What is a Manifold?

- A **manifold** is a lower-dimensional, smoothly curved surface embedded within a higher-dimensional space. In machine learning, a **data manifold** is an assumption that real-world high-dimensional data (like images, speech, or text) actually lies on a lower-dimensional structure within the high-dimensional space.

For example:

- A grayscale **28×28 image (784 dimensions)** of digits can be represented using far fewer meaningful features (edges, strokes, curves, etc.).
 - Faces, speech, and other structured data can be represented using a smaller set of intrinsic variables (e.g., pose, lighting, identity).
- Manifold learning aims to **extract this low-dimensional structure** while preserving meaningful relationships within the data.

Learning Manifold with Autoencoders

- Data tends to concentrate around **low-dimensional manifolds**.
- Autoencoders learn **not only to reconstruct data** but also to **capture the manifold structure**.
- Other ML algorithms only learn to perform well on the manifold but **may fail on out-of-manifold inputs**.
- **Manifolds**: Low-dimensional structures embedded in high-dimensional space.
- Defined by **tangent planes** at each point, representing **local directions of variation**.
- Autoencoders learn representations that **capture these tangent planes**.

Learning Manifold with Autoencoders

Training Autoencoders for Manifold Learning

Autoencoder training involves two forces:

1. **Reconstructing the input** from a hidden representation.
2. **Regularization constraints** (e.g., weight penalties, sparsity) to avoid **identity mapping**.
 - This forces the model to **capture the essential data structure** instead of memorizing input.

Autoencoders vs. Other Manifold Learning Methods

- **Autoencoders learn a general function (encoder-decoder)**, mapping any input to its manifold representation.
- **Non-parametric methods (e.g., nearest-neighbor graph)** associate each data point with a tangent plane.
- **Limitation of non-parametric methods:** Require many samples to generalize, struggle with complex manifolds.

Learning Manifold with Autoencoders

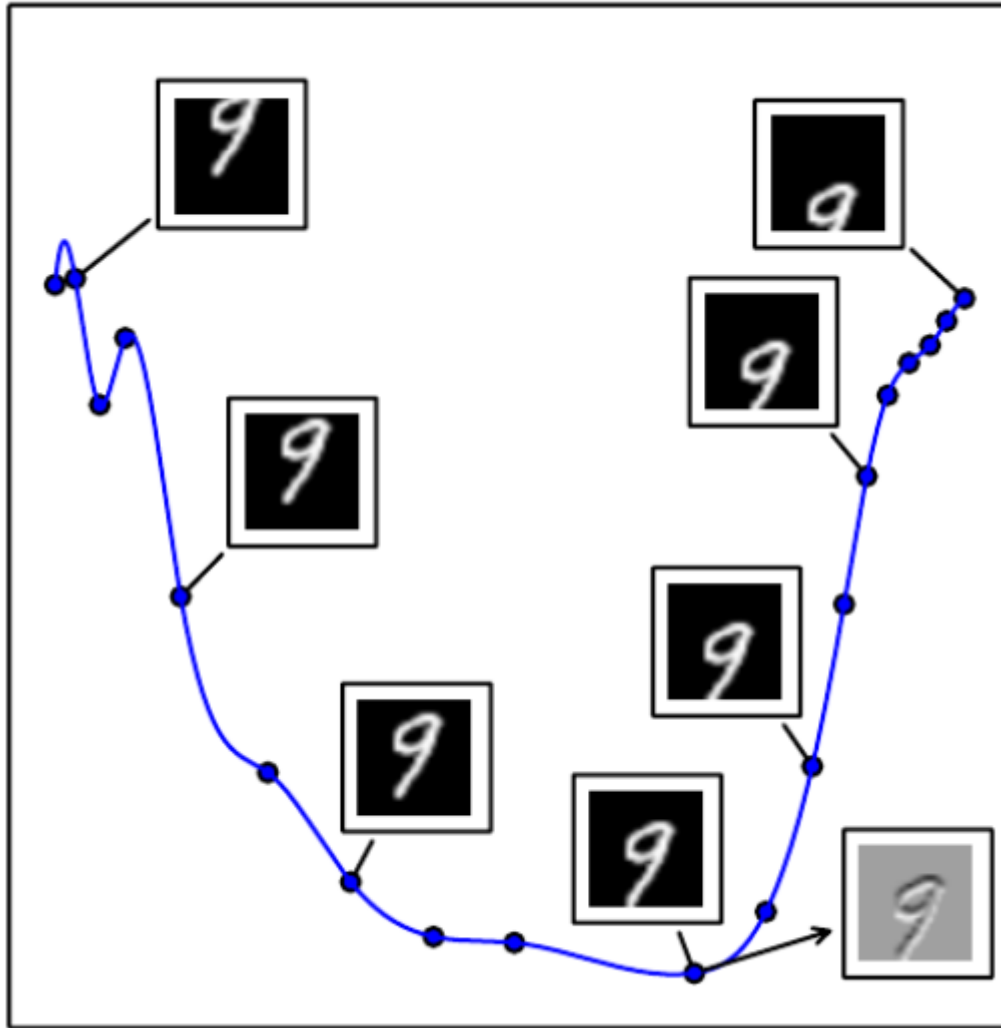


Figure 14.6: An illustration of the concept of a tangent hyperplane. Here we create a one-dimensional manifold in 784-dimensional space. We take an MNIST image with 784 pixels and transform it by translating it vertically. The amount of vertical translation defines a coordinate along a one-dimensional manifold that traces out a curved path through image space. This plot shows a few points along this manifold. For visualization, we have projected the manifold into two dimensional space using PCA. An n -dimensional manifold has an n -dimensional tangent plane at every point. This tangent plane touches the manifold exactly at that point and is oriented parallel to the surface at that point. It defines the space of directions in which it is possible to move while remaining on the manifold. This one-dimensional manifold has a single tangent line. We indicate an example tangent line at one point, with an image showing how this tangent direction appears in image space. Gray pixels indicate pixels that do not change as we move along the tangent line, white pixels indicate pixels that brighten, and black pixels indicate pixels that darken.

Learning Manifold with Autoencoders

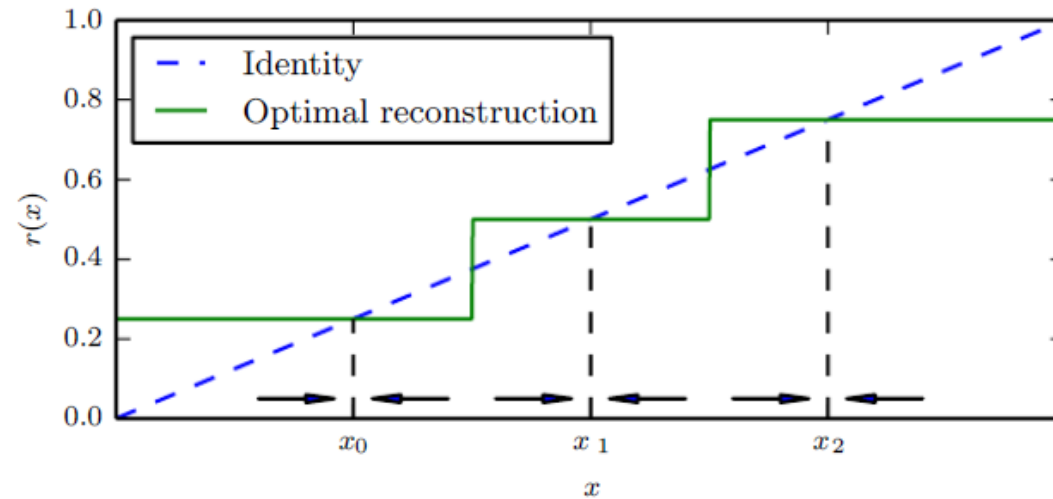


Figure 14.7: If the autoencoder learns a reconstruction function that is invariant to small perturbations near the data points, it captures the manifold structure of the data. Here the manifold structure is a collection of 0-dimensional manifolds. The dashed diagonal line indicates the identity function target for reconstruction. The optimal reconstruction function crosses the identity function wherever there is a data point. The horizontal arrows at the bottom of the plot indicate the $r(\mathbf{x}) - \mathbf{x}$ reconstruction direction vector at the base of the arrow, in input space, always pointing towards the nearest “manifold” (a single datapoint, in the 1-D case). The denoising autoencoder explicitly tries to make the derivative of the reconstruction function $r(\mathbf{x})$ small around the data points. The contractive autoencoder does the same for the encoder. Although the derivative of $r(\mathbf{x})$ is asked to be small around the data points, it can be large between the data points. The space between the data points corresponds to the region between the manifolds, where the reconstruction function must have a large derivative in order to map corrupted points back onto the manifold.

Learning Manifold with Autoencoders

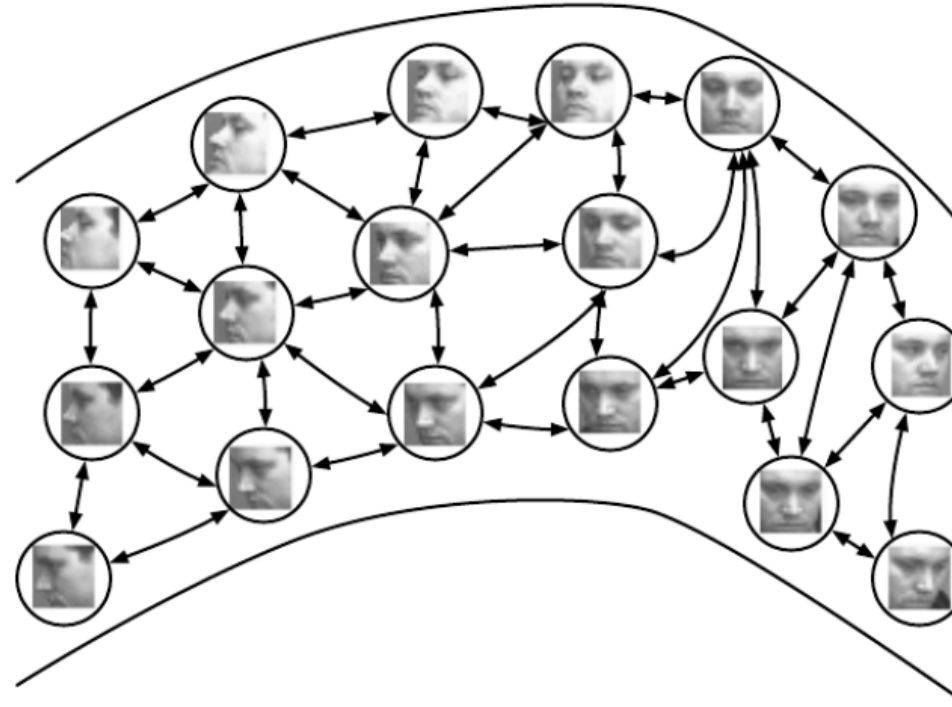


Figure 14.8: Non-parametric manifold learning procedures build a nearest neighbor graph in which nodes represent training examples a directed edges indicate nearest neighbor relationships. Various procedures can thus obtain the tangent plane associated with a neighborhood of the graph as well as a coordinate system that associates each training example with a real-valued vector position, or **embedding**. It is possible to generalize such a representation to new examples by a form of interpolation. So long as the number of examples is large enough to cover the curvature and twists of the manifold, these approaches work well. Images from the QMUL Multiview Face Dataset (Gong *et al.*, 2000).

Learning Manifold with Autoencoders

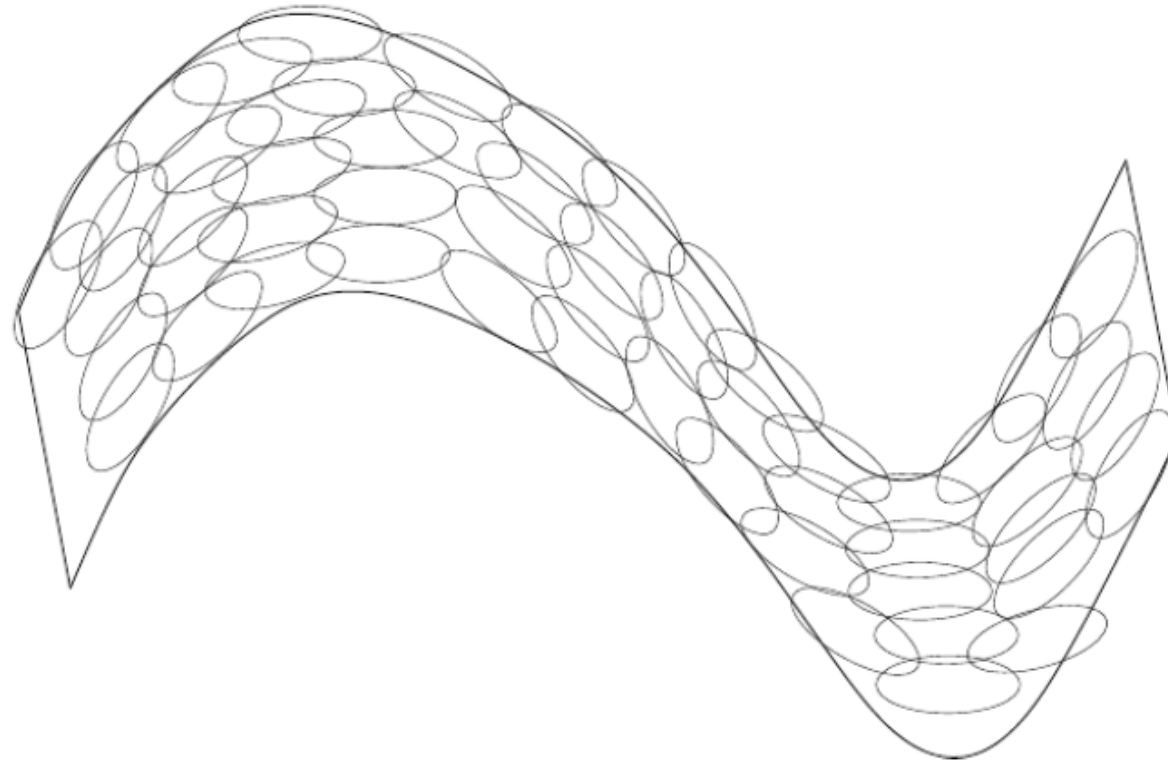


Figure 14.9: If the tangent planes (see figure 14.6) at each location are known, then they can be tiled to form a global coordinate system or a density function. Each local patch can be thought of as a local Euclidean coordinate system or as a locally flat Gaussian, or “pancake,” with a very small variance in the directions orthogonal to the pancake and a very large variance in the directions defining the coordinate system on the pancake. A mixture of these Gaussians provides an estimated density function, as in the manifold Parzen window algorithm (Vincent and Bengio, 2003) or its non-local neural-net based variant (Bengio *et al.*, 2006c).

Learning Manifold with Autoencoders

Challenges in Manifold Learning

- Manifolds in real-world AI problems are **highly complex, twisted, and nonlinear**.
- **Local methods struggle** to generalize beyond seen data.
- Example: Image translation creates **complex manifolds** that simple interpolation cannot capture.

Deep Learning for Manifold Learning

- **Deep learning overcomes limitations** by using **distributed representations**.
- **Autoencoders + deep networks** capture **highly non-linear** and **structured** manifolds.
- Essential for **unsupervised representation learning** in AI applications.

Manifold Learning Pros and Cons

Pros:

- Captures meaningful **low-dimensional structure** from high-dimensional data.
- Can be used for **data compression, denoising, and visualization**.
- Helps in **unsupervised feature learning** and pretraining deep models.
- Variants like VAEs allow **probabilistic sampling** from the manifold.

Cons:

- Needs a **carefully designed bottleneck layer** to avoid losing important information.
- Sensitive to **hyperparameter tuning** (e.g., learning rate, architecture depth).
- Denoising and contractive autoencoders **require additional regularization** to prevent trivial identity mapping.

Contractive Autoencoders

- Contractive autoencoders (CAEs) are a variant of autoencoders that add a regularization term to the standard reconstruction loss.
- This extra term is based on the Jacobian of the encoder's mapping with respect to the input.
- By penalizing the (squared) Frobenius norm of this Jacobian, the model is encouraged to learn representations that are locally insensitive—that is, small changes in the input produce only small changes in the hidden representation.

Note:

- **Robust Representations:** By penalizing the Jacobian norm, the autoencoder learns representations that change very little for small changes in the input, leading to robust and invariant features.
- **Balancing Act:** The hyperparameter λ determines how strongly the model emphasizes robustness versus reconstruction accuracy.

Contractive Autoencoders

In mathematical terms, if the encoder maps an input x to a latent representation h via

$$h = f(x),$$

then the CAE's loss function is given by:

$$L(x, \hat{x}) = \|x - \hat{x}\|^2 + \lambda \|J_f(x)\|_F^2,$$

where:

- $\|x - \hat{x}\|^2$ is the reconstruction error (for example, using mean squared error),
- $J_f(x)$ is the Jacobian matrix of $f(x)$ with respect to x (i.e. it contains all first-order partial derivatives $\partial h_i / \partial x_j$),
- $\|J_f(x)\|_F^2$ denotes the squared Frobenius norm (the sum of squares of all elements in the Jacobian), and
- λ is a hyperparameter controlling the strength of the penalty.

This contractive penalty forces the mapping to be “flat” (or contractive) in the neighborhood of training samples, making the learned features robust to small perturbations of the input.

Contractive Autoencoders

Contractive autoencoders focus on penalizing the Jacobian of the encoder because this first-order derivative directly measures how sensitive the hidden representation is to small changes in the input. In other words, the Jacobian tells us how much a tiny perturbation in the input will affect the features learned by the encoder.

Why the Jacobian is used:

- **Local Sensitivity:** The Jacobian captures the local, linear sensitivity (first-order effects) of the representation with respect to the input. Penalizing its Frobenius norm forces the encoder to be locally invariant, meaning small changes in x lead to only small changes in $h=f(x)$.
- **Computational Efficiency:** Calculating the Jacobian is relatively straightforward (especially when the activation functions are elementwise) and can be efficiently computed using automatic differentiation. In contrast, higher-order derivatives (like the Hessian) are computationally expensive and complex to compute, with diminishing returns in capturing additional invariance.
- **Sufficiency for Robustness:** In many cases, controlling the first-order sensitivity is sufficient to yield robust features. While some variants (like CAE+H) consider approximations of higher order derivatives, the dominant and most informative term is the Jacobian, which ensures the representation is stable in the immediate neighborhood of each training sample.

Thus, by penalizing only the Jacobian, the autoencoder efficiently encourages the model to learn invariant, robust features without incurring the high cost of computing and optimizing over higher-order derivatives.

Predictive Sparse Decomposition (PSD)

PSD builds on sparse coding by adding a feed-forward encoder (Parametric autoencoders)

- **Goal:** Learn a sparse representation $\mathbf{h}$ for input $\mathbf{x}$
- **Key Idea:** Train an encoder $f(\mathbf{x})$ to predict an approximation $\hat{\mathbf{h}}$ to the optimal sparse code
- **Objective:** Combine reconstruction error, sparsity penalty, and prediction error

Predictive Sparse Decomposition (PSD) – Objective Function

Traditional Sparse Coding Cost:

$$E(h) = \frac{1}{2} \|x - g(h)\|^2 + \lambda \|h\|_1$$

PSD Extended Loss:

$$L(x, h; g, f) = \frac{1}{2} \|x - g(h)\|^2 + \lambda \|h\|_1 + \gamma \|h - f(x)\|^2$$

Interpretation:

- Reconstruction term: $\frac{1}{2} \|x - g(h)\|^2$
- Sparsity term: $\lambda \|h\|_1$
- Prediction alignment: $\gamma \|h - f(x)\|^2$

Predictive Sparse Decomposition (PSD)

Training Process

Alternate Optimization Steps:

1. **Sparse Coding Step:** For each x , solve for the optimal sparse code h^* by minimizing $L(h)$
2. **Parameter Update:** Update dictionary g and encoder $f(x)$ parameters (e.g., W and b) via gradient descent

Outcome: Encoder learns to rapidly predict h^* without iterative optimization at test time

Dept. of Machine

Predictive Sparse Decomposition (PSD)

Training Process

Alternating Minimization:

- Like sparse coding, PSD training alternates between:
 - Minimizing w.r.t. the latent code h
 - Minimizing w.r.t. the model parameters

Fast Minimization for h :

- $f(x)$ provides a strong initialization for h
- Cost function keeps h near $f(x)$
- Reasonable h can be obtained in as few as ten gradient descent steps

PSD Training Vs Traditional Sparse coding

Joint Training Approach:

- PSD simultaneously regularizes the decoder so that $f(x)$ infers good code values
- Unlike first training sparse coding then training $f(x)$ separately, PSD integrates both steps

Learned Approximate Inference:

- PSD is an example of learning approximate inference via a differentiable encoder

PSD in Practice and Interpretation

During Training: Iterative optimization is used to compute h

At Deployment: Only the parametric encoder $f(x)$ is evaluated (computationally inexpensive)

Stacking & Deep Initialization:

- As a differentiable function, $f(x)$ enables stacking PSD models
- Can be used to initialize deep networks for further training with alternative criteria

Probabilistic Perspective:

- PSD can be seen as training a directed sparse coding probabilistic model
- Maximizes a lower bound on the log-likelihood (see Section 19.5 for details)

Predictive Sparse Decomposition (PSD)

Training Process

Hyperparameters: $\lambda = 0.1, \gamma = 0.5$

Input Vector: $x = \begin{bmatrix} 2 \\ 3 \end{bmatrix}$

Dictionary g :

Assume a simple 2×2 dictionary given by $g = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$

(For simplicity, we take g as the identity so that $g(h)$ is just h .)

Predictive Encoder $f(x)$: $f(x) = Wx + b$, $W = \begin{bmatrix} 0.5 & 0 \\ 0 & 0.5 \end{bmatrix}$, $b = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$
Linear mapping

$$f(x) = \begin{bmatrix} 1 \\ 1.5 \end{bmatrix}$$

Predictive Sparse Decomposition (PSD)

Training Process

Solving for the Optimal Sparse Code h (Dimension 1)

For h_1 :

$$L_1(h_1) = \frac{1}{2}(2 - h_1)^2 + 0.1 |h_1| + 0.5 (h_1 - 1)^2$$

- Expand and combine:

$$L_1(h_1) = h_1^2 - 2.9h_1 + 2.5$$

- Set derivative to 0:

$$2h_1 - 2.9 = 0 \Rightarrow h_1 = 1.45$$

Optimal Code: $h^* = \begin{bmatrix} 1.45 \\ 2.2 \end{bmatrix}$

Solving for h (Dimension 2) & Loss Components

For h_2 :

$$L_2(h_2) = \frac{1}{2}(3 - h_2)^2 + 0.1 |h_2| + 0.5 (h_2 - 1.5)^2$$

- Expand and combine:

$$L_2(h_2) = h_2^2 - 4.4h_2 + 5.625$$

- Set derivative to 0:

$$2h_2 - 4.4 = 0 \Rightarrow h_2 = 2.2$$

Predictive Sparse Decomposition (PSD)

Training Process

Loss Breakdown:

- Reconstruction Error: $\frac{1}{2} \|x - g(h^*)\|^2 = \frac{1}{2} \|(2, 3)^T - (1.45, 2.2)^T\|^2 \approx 0.47125$
- Sparsity Penalty: $0.1(1.45 + 2.2) = 0.365$
- Prediction Error: $0.5 \|h^* - f(x)\|^2 \approx 0.34625$
- Total Loss: ≈ 1.1825

Predictive Sparse Decomposition (PSD)

How does PSD uses the Loss in Training?

Step 1: For each input x , compute the optimal sparse code h^* by minimizing:

$$L(x, h; g, f) = \frac{1}{2} \|x - g(h)\|^2 + \lambda \|h\|_1 + \gamma \|h - f(x)\|^2.$$

Step 2: Update the dictionary g and encoder parameters (of f) using gradient descent on the overall loss

Result:

- The encoder $f(x)$ learns to rapidly predict a good approximation of h^*
- At test time, feed-forward evaluation $f(x)$ replaces iterative optimization

Applications of Autoencoders

- Autoencoders have been successfully applied to tasks like dimensionality reduction and information retrieval.
- Dimensionality reduction was one of the first motivations for studying autoencoders and deep learning.
- Early work (e.g., Hinton & Salakhutdinov, 2006) showed deep autoencoders can yield better low-dimensional representations than PCA.

Applications of Autoencoders

Dimensionality Reduction via Deep Autoencoders

Example:

- Hinton & Salakhutdinov (2006) trained a stack of RBMs.
- Initialized a deep autoencoder with gradually smaller hidden layers, ending in a 30-unit bottleneck.

Outcome:

- Lower reconstruction error compared to PCA (with 30 dimensions).
- Learned representation forms well-separated clusters that align with underlying categories.

Applications of Autoencoders

Benefits of Lower – Dimensional Representations

- **Improved Performance:** Boosts classification accuracy by grouping semantically related examples.
- **Efficiency:** Models with smaller representations consume less memory and have faster runtime.
- **Generalization:** The mapping hints at inherent structure, aiding in generalization (Salakhutdinov & Hinton, 2007b; Torralba et al., 2008).

Applications of Autoencoders

Autoencoders in Information Retrieval

Information Retrieval Task:

- Find entries in a database that resemble a query entry.

Advantage:

- Low-dimensional codes allow for efficient search, sometimes using hash tables.
- Binary codes map naturally to hash keys.

Applications of Autoencoders

Semantic Hashing

Concept:

- Train the dimensionality reduction algorithm to produce low-dimensional, binary codes.

How It Works:

- Use sigmoids on the final layer to force outputs toward 0 or 1.
- Inject additive noise before the sigmoid during training; gradually increase noise so that the network learns to amplify its inputs for saturation.

Benefits:

- Database entries can be stored in a hash table mapped by binary codes.
- Efficient retrieval by exact match or by flipping individual bits for near matches.

Applications of Autoencoders

Further Developments

Optimizing Retrieval:

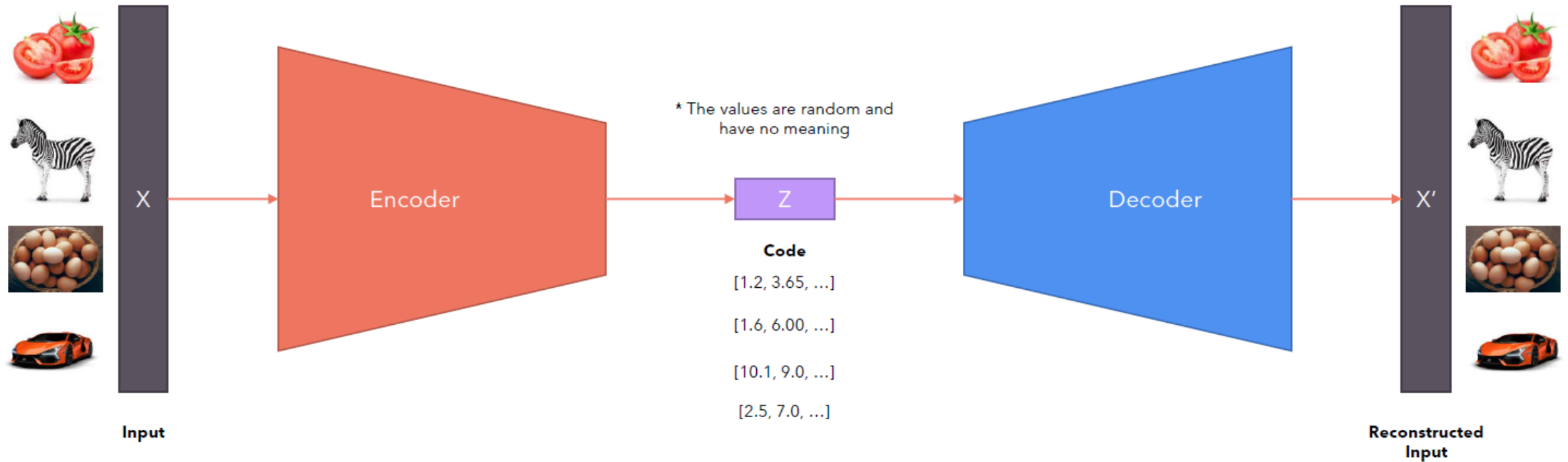
- Subsequent work (e.g., Norouzi and Fleet, 2011) has explored training the representations to directly optimize retrieval performance.

Applications:

- Semantic hashing has been applied to both text (Salakhutdinov & Hinton, 2007b, 2009b) and images (Torralba et al., 2008; Weiss et al., 2008; Krizhevsky & Hinton, 2011).

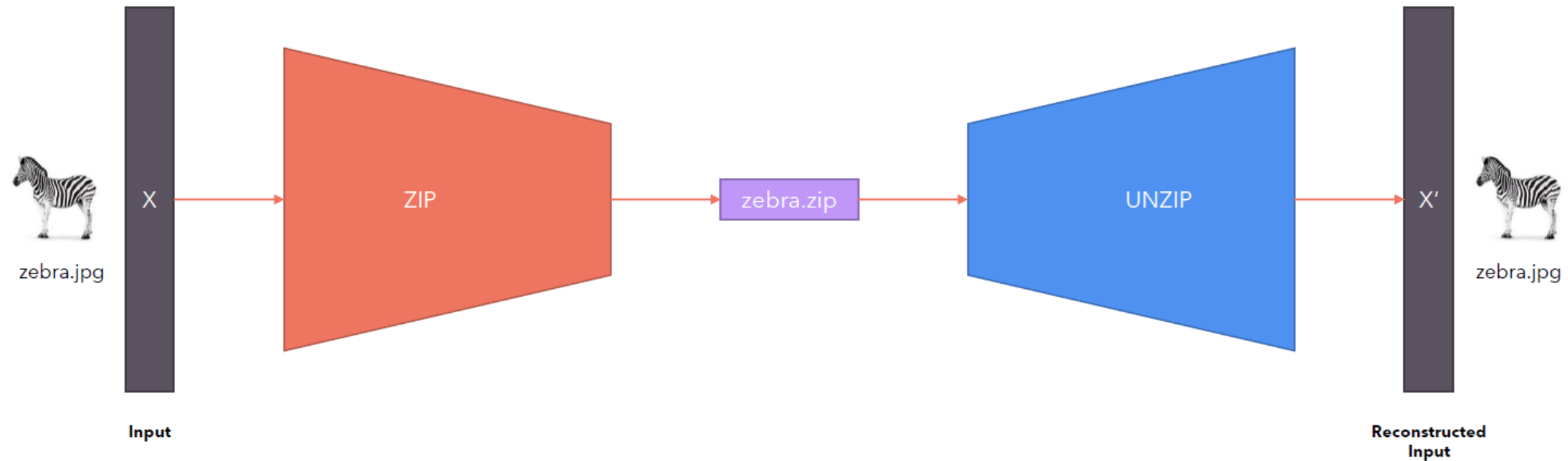
Variational Autoencoders (VAEs)

What is an Autoencoder?



Variational Autoencoders (VAEs)

Analogy with file compression



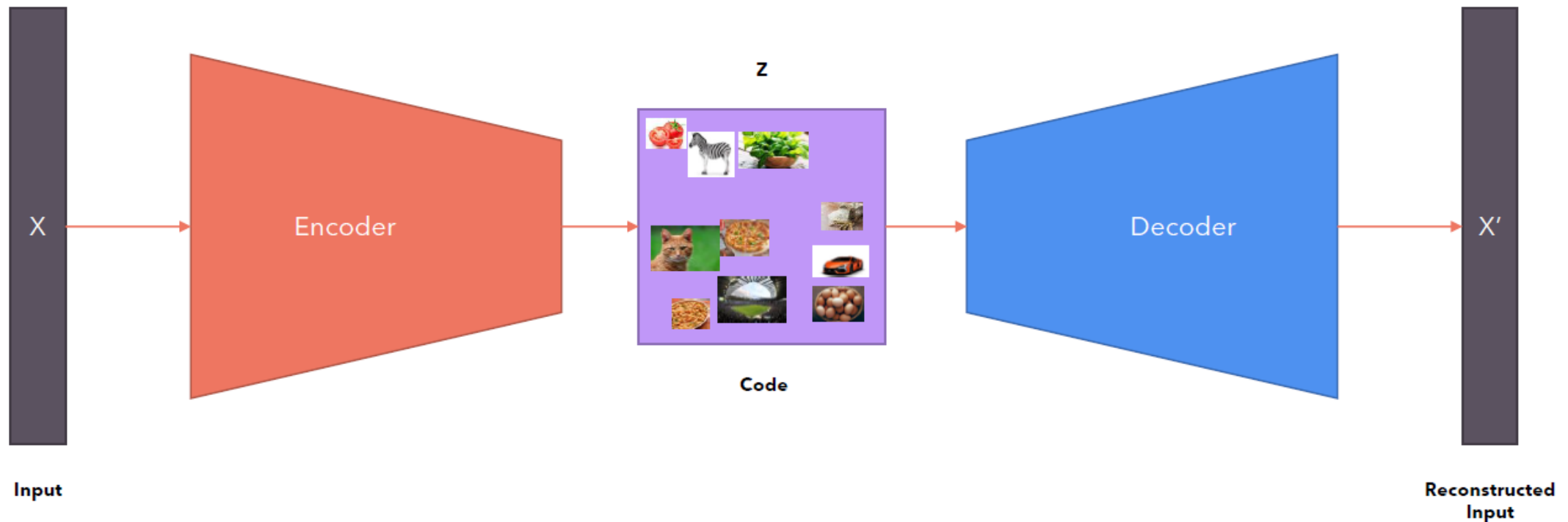
In a Best Autoencoder

- The code should be as small as possible, that is, the dimension of the Z vector should be as small as possible.
- The reconstructed input should be as close as possible to the original input.

Variational Autoencoders (VAEs)

What's the problem with Autoencoders?

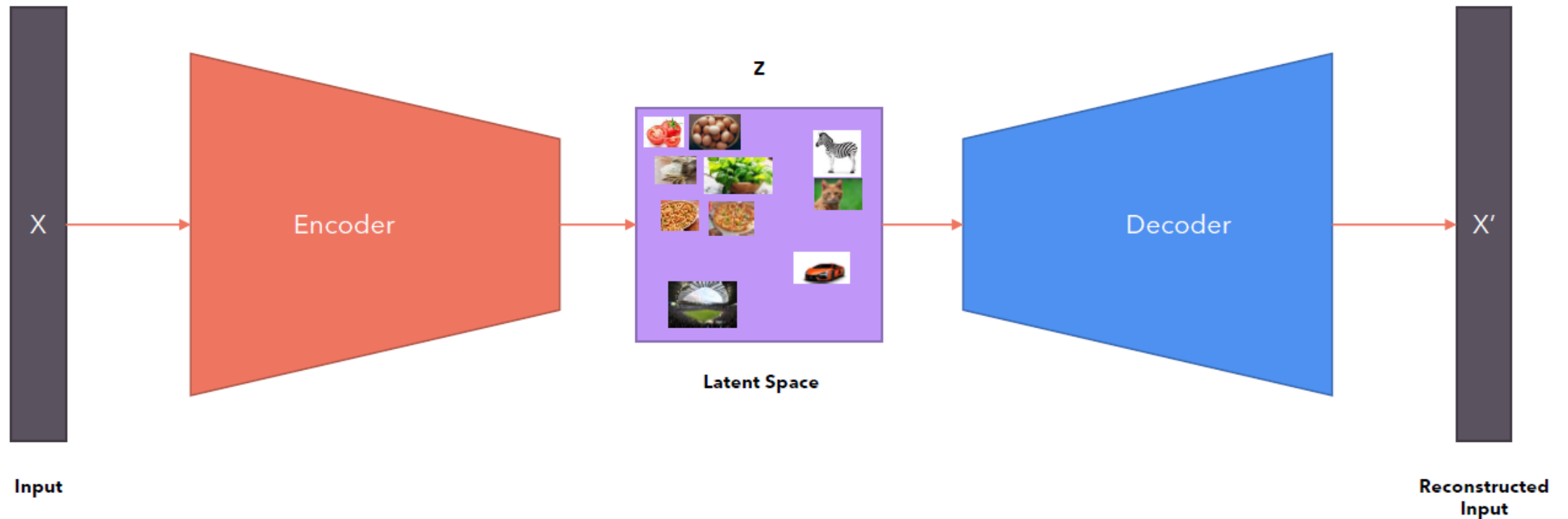
The code learned by the model **makes no sense**. That is, the model can just assign any vector to the inputs without the numbers in the vector representing any pattern. The model doesn't capture any **semantic relationship** between the data.



Variational Autoencoders (VAEs)

Introducing the Variational Autoencoder

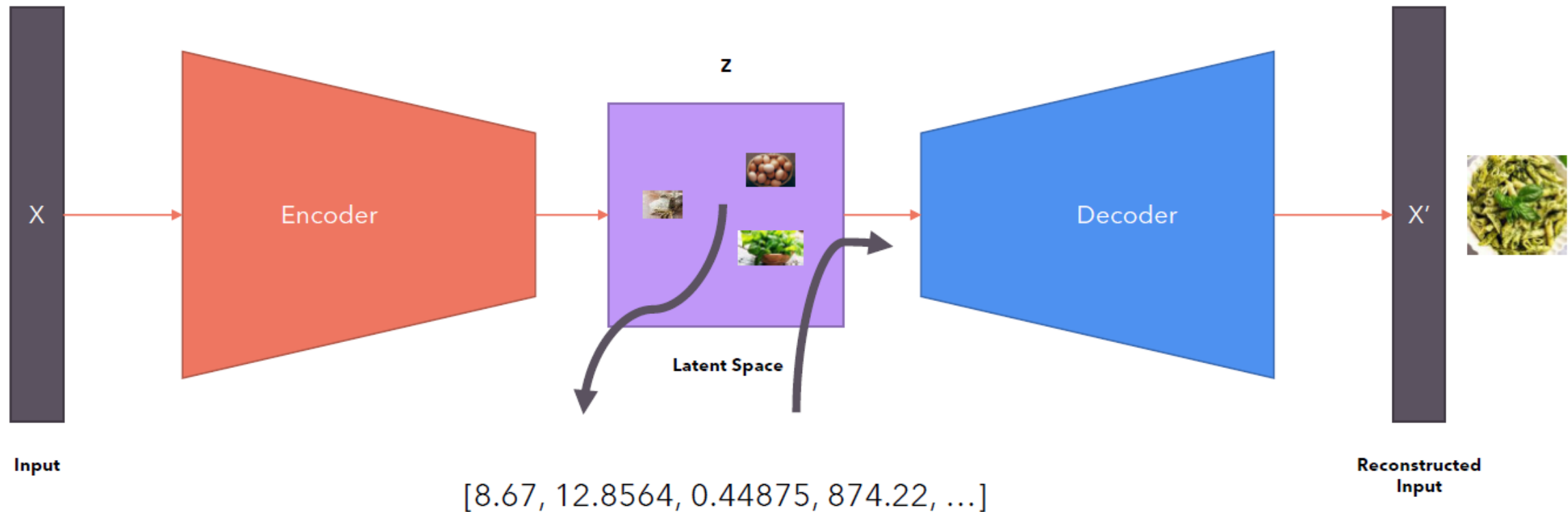
The variational autoencoder, instead of learning a code, learns a "**latent space**". The latent space represents the parameters of a (multivariate) distribution.



Variational Autoencoders (VAEs)

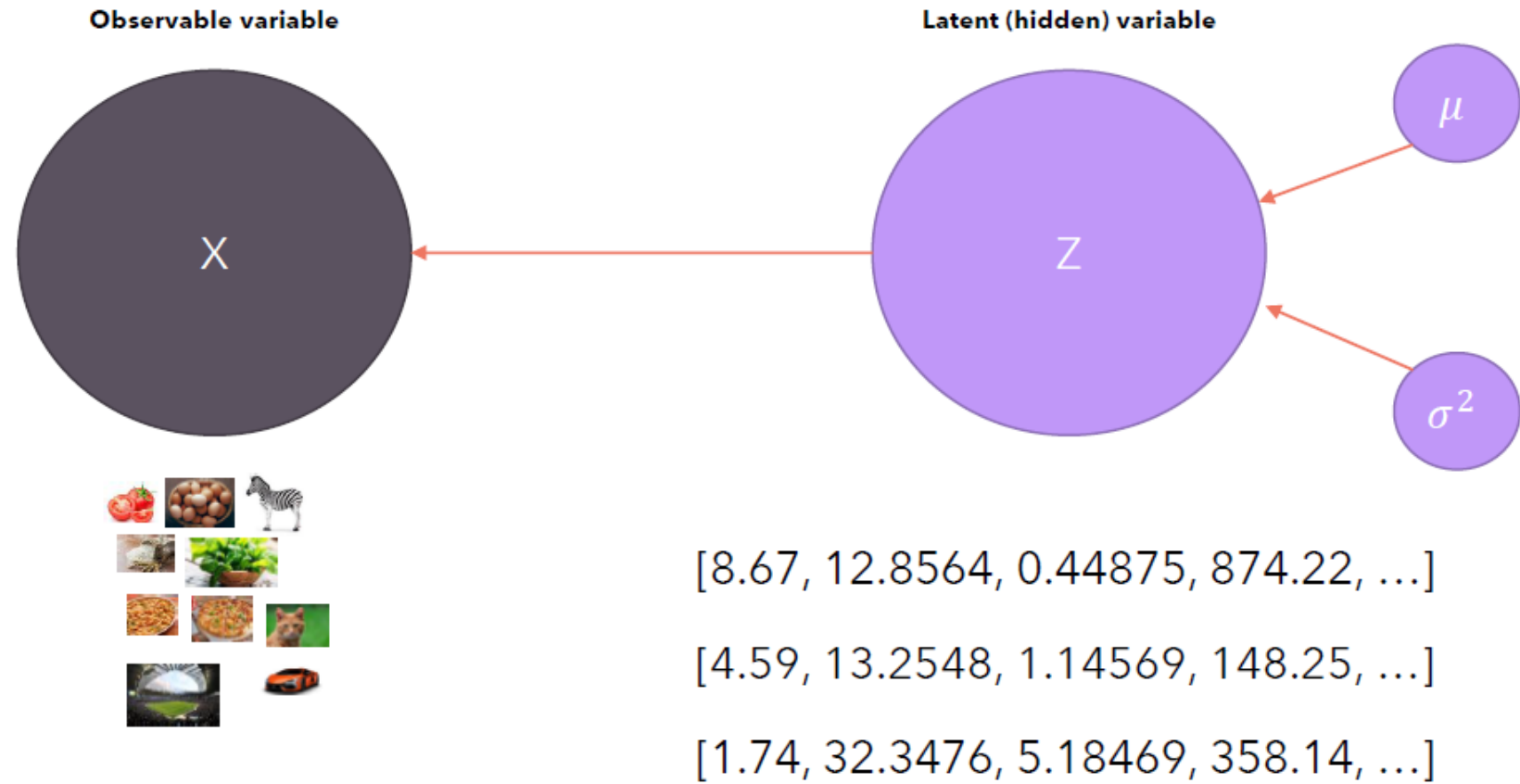
Sampling the latent space

Just like when you use Python to generate a random number between 1 and 100, you're sampling from a uniform (pseudo)random distribution between 1 and 100. In the same way, we can sample from the latent space in order to generate a random vector, give it to the decoder and generate new data.



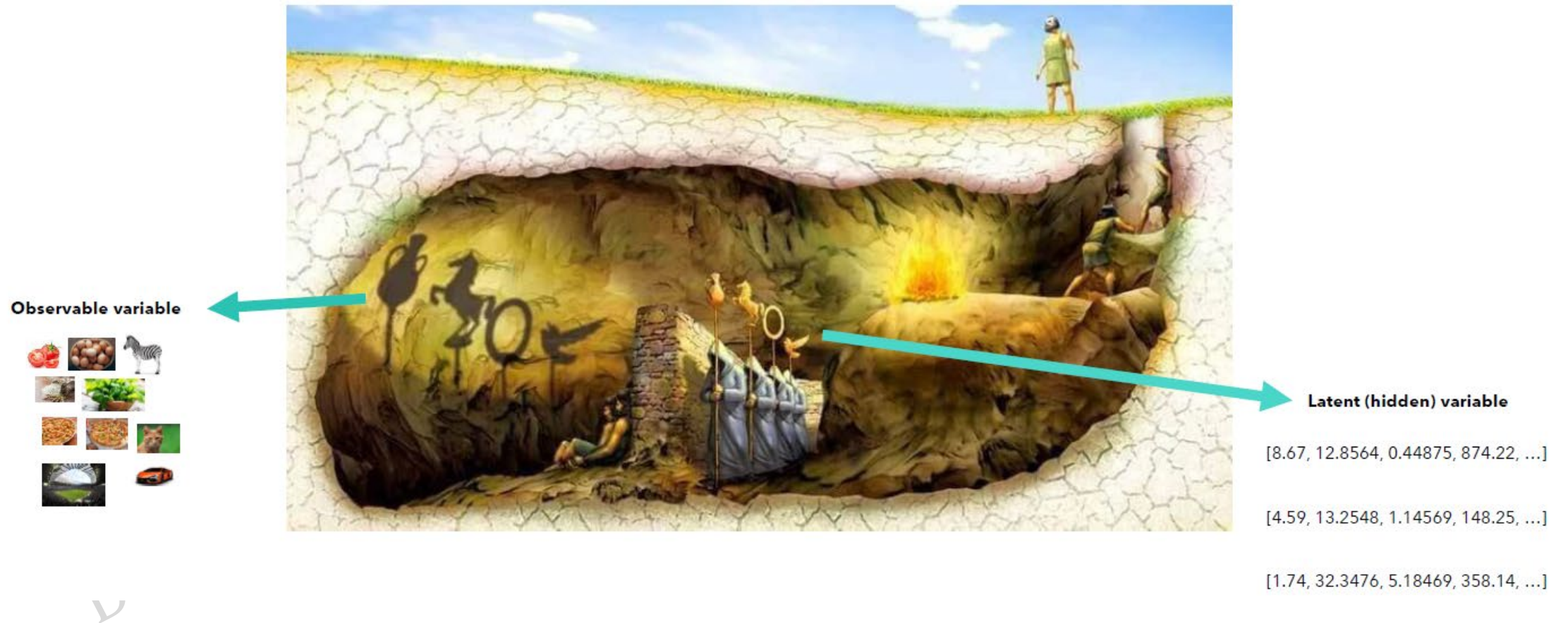
Variational Autoencoders (VAEs)

Why is it called latent space?



Variational Autoencoders (VAEs)

Plato's allegory of the cave



Variational Autoencoders (VAEs)

Kullback-Leibler Divergence

Expectation of a random variable

$$E_x[f(x)] = \int xf(x)dx$$

Chain rule of probability

$$P(x, y) = P(x|y)P(y)$$

Bayes' Theorem

$$P(x | y) = \frac{P(y|x)P(x)}{P(y)}$$

$$D_{KL}(P\|Q) = \int p(x) \log\left(\frac{p(x)}{q(x)}\right) dx$$

Properties:

- Not symmetric.
- Always ≥ 0
- It is equal to 0 if and only if $P = Q$

Variational Autoencoders (VAEs)

Let's define our model

We can define the likelihood of our data as the marginalization over the joint probability with respect to the latent variable

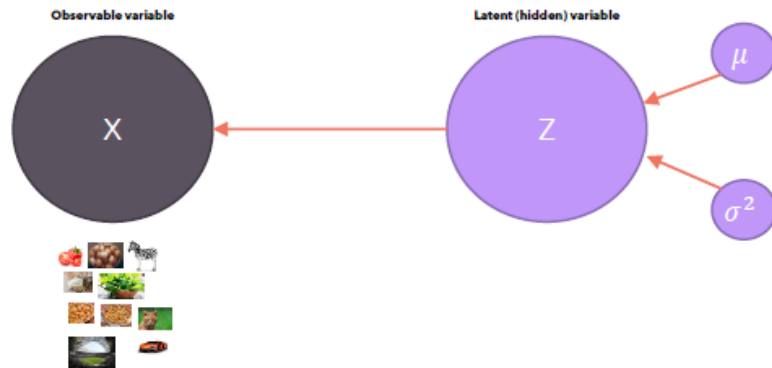
$$p(\mathbf{x}) = \int p(\mathbf{x}, \mathbf{z}) d\mathbf{z}$$

Is **intractable** because we would need to evaluate this integral over all latent variables $\mathbf{Z}$.

... or we can use the Chain rule of probability

$$p(\mathbf{x}) = \frac{p(\mathbf{x}, \mathbf{z})}{p(\mathbf{z}|\mathbf{x})}$$

We **don't** have a ground truth $p(\mathbf{z}|\mathbf{x})$
... which is also what we're trying to find!



Intractable problem = a problem that can be solved in theory (e.g. given large but finite resources, especially time), but for which in practice any solution takes too many resources to be useful, is known as an intractable problem.

Variational Autoencoders (VAEs)

A chicken and egg problem

In order to have a tractable $p(\mathbf{x})$ we need a tractable $p(\mathbf{z}|\mathbf{x})$

$$p(\mathbf{x}) = \frac{p(\mathbf{x}, \mathbf{z})}{p(\mathbf{z}|\mathbf{x})}$$



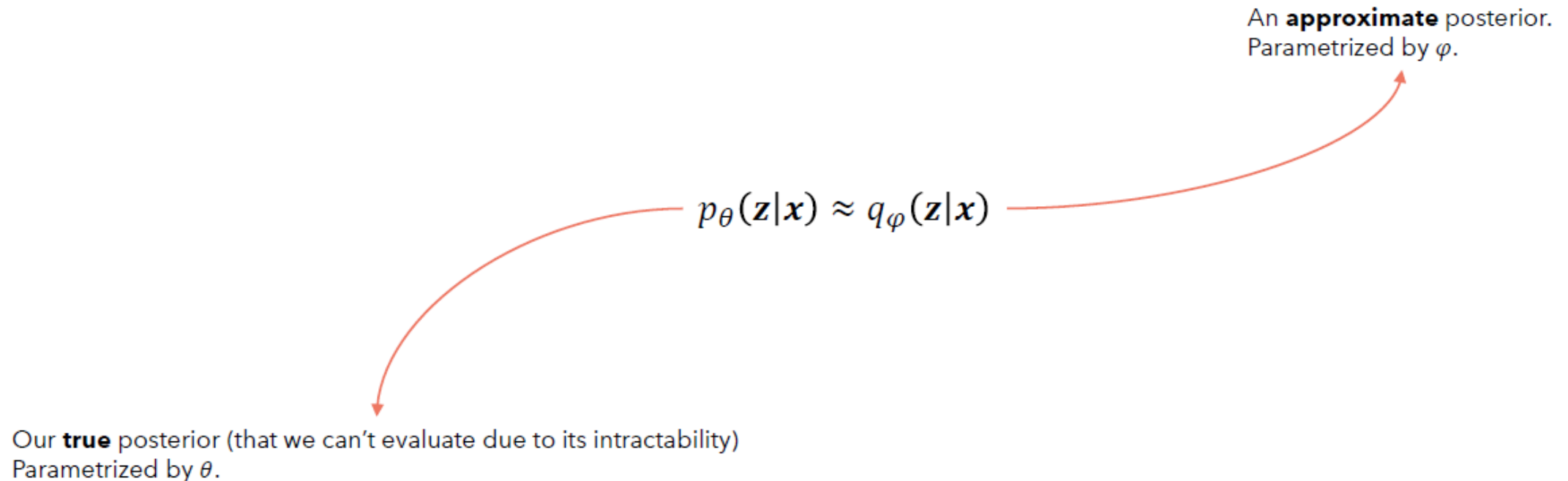
$$p(\mathbf{z}|\mathbf{x}) = \frac{p(\mathbf{x}, \mathbf{z})}{p(\mathbf{x})}$$

In order to have a tractable $p(\mathbf{z}|\mathbf{x})$ we need a tractable $p(\mathbf{x})$



Variational Autoencoders (VAEs)

Can we find a surrogate?



Variational Autoencoders (VAEs)

$$\log p_{\theta}(x) = \log p_{\theta}(x)$$

$$= \log p_{\theta}(x) \int q_{\varphi}(z|x) dz$$

Multiply by 1

$$= \int \log p_{\theta}(x) q_{\varphi}(z|x) dz$$

Bring inside the integral

$$= E_{q_{\varphi}(z|x)}[\log p_{\theta}(x)]$$

Definition of expectation

$$= E_{q_{\varphi}(z|x)} \left[\log \frac{p_{\theta}(x, z)}{p_{\theta}(z|x)} \right]$$

Apply the equation $p_{\theta}(x) = \frac{p_{\theta}(x, z)}{p_{\theta}(z|x)}$

$$= E_{q_{\varphi}(z|x)} \left[\log \frac{p_{\theta}(x, z) q_{\varphi}(z|x)}{p_{\theta}(z|x) q_{\varphi}(z|x)} \right]$$

Multiply by 1

$$= E_{q_{\varphi}(z|x)} \left[\log \frac{p_{\theta}(x, z)}{q_{\varphi}(z|x)} \right] + E_{q_{\varphi}(z|x)} \left[\log \frac{q_{\varphi}(z|x)}{p_{\theta}(z|x)} \right]$$

Split the expectation

$$= E_{q_{\varphi}(z|x)} \left[\log \frac{p_{\theta}(x, z)}{q_{\varphi}(z|x)} \right] + \underbrace{D_{KL}(q_{\varphi}(z|x) \| p_{\theta}(z|x))}_{\geq 0}$$

Definition of KL divergence

Variational Autoencoders (VAEs)

What can we infer?

$$\log p_{\theta}(\mathbf{x}) = \underbrace{E_{q_{\varphi}(\mathbf{z}|\mathbf{x})} \left[\log \frac{p_{\theta}(\mathbf{x}, \mathbf{z})}{q_{\varphi}(\mathbf{z}|\mathbf{x})} \right]}_{\text{ELBO}} + \underbrace{D_{KL}(q_{\varphi}(\mathbf{z}|\mathbf{x}) \| p_{\theta}(\mathbf{z}|\mathbf{x}))}_{\geq 0}$$

ELBO = Evidence Lower Bound

$$\text{Total Compensation} = \text{Base Salary} + \underbrace{\text{Bonus}}_{\geq 0}$$

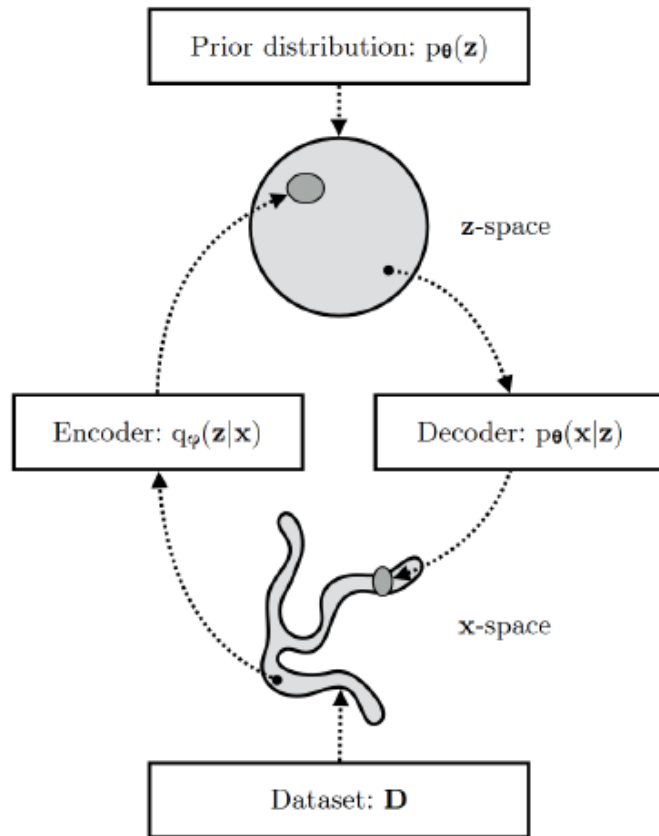
We can for sure deduce the following:

$$\text{Total Compensation} \geq \text{Base Salary}$$

$$\log p_{\theta}(\mathbf{x}) \geq E_{q_{\varphi}(\mathbf{z}|\mathbf{x})} \left[\log \frac{p_{\theta}(\mathbf{x}, \mathbf{z})}{q_{\varphi}(\mathbf{z}|\mathbf{x})} \right]$$

Variational Autoencoders (VAEs)

ELBO in detail



$$\log p_\theta(\mathbf{x}) \geq E_{q_\phi(\mathbf{z}|\mathbf{x})} \left[\log \frac{p_\theta(\mathbf{x}, \mathbf{z})}{q_\phi(\mathbf{z}|\mathbf{x})} \right] = E_{q_\phi(\mathbf{z}|\mathbf{x})} \left[\log \frac{p_\theta(\mathbf{x}|\mathbf{z})p(\mathbf{z})}{q_\phi(\mathbf{z}|\mathbf{x})} \right]$$

Chain rule of probability

$$= E_{q_\phi(\mathbf{z}|\mathbf{x})} [\log p_\theta(\mathbf{x}|\mathbf{z})] + E_{q_\phi(\mathbf{z}|\mathbf{x})} \left[\log \frac{p(\mathbf{z})}{q_\phi(\mathbf{z}|\mathbf{x})} \right]$$

Split the expectation

$$= E_{q_\phi(\mathbf{z}|\mathbf{x})} [\log p_\theta(\mathbf{x}|\mathbf{z})] - D_{KL}(q_\phi(\mathbf{z}|\mathbf{x}) \| p_\theta(\mathbf{z}))$$

Definition of KL divergence

$$\underbrace{\phantom{E_{q_\phi(\mathbf{z}|\mathbf{x})} [\log p_\theta(\mathbf{x}|\mathbf{z})]}}_{\text{Profit}} = \underbrace{\phantom{E_{q_\phi(\mathbf{z}|\mathbf{x})} [\log p_\theta(\mathbf{x}|\mathbf{z})]}}_{\text{Revenue}} - \underbrace{\phantom{D_{KL}(q_\phi(\mathbf{z}|\mathbf{x}) \| p_\theta(\mathbf{z}))}}_{\text{Costs}}$$

Maximizing the ELBO means:

1. Maximizing the first term: maximizing the reconstruction likelihood of the decoder
2. Minimizing the second term: minimizing the distance between the learned distribution and the prior belief we have over the latent variable.

Variational Autoencoders (VAEs)

Maximizing the ELBO: A little introduction to estimators

- When we have a function we want to **maximize**, we usually take the gradient and adjust the weights of the model so that they move **along** the gradient direction.
- When we have a function we want to **minimize**, we usually take the gradient, and adjust the weights of the model so that they move **against** the gradient direction.

Stochastic Gradient Descent

When used to minimize the above function, a standard (or "batch") **gradient descent** method would perform the following iterations:

$$w := w - \eta \nabla Q(w) = w - \frac{\eta}{n} \sum_{i=1}^n \nabla Q_i(w),$$

where η is a step size (sometimes called the **learning rate** in machine learning).

SGD is **stochastic** because we choose the minibatch randomly from our dataset and we then average the loss over the minibatch

$$L(\theta, \varphi, \mathbf{x}) = E_{q_{\varphi}(\mathbf{z}|\mathbf{x})} \left[\log \frac{p_{\theta}(\mathbf{x}, \mathbf{z})}{q_{\varphi}(\mathbf{z}|\mathbf{x})} \right] = E_{q_{\varphi}(\mathbf{z}|\mathbf{x})} [\log p_{\theta}(\mathbf{x}|\mathbf{z})] - D_{KL} (q_{\varphi}(\mathbf{z}|\mathbf{x}) \| p_{\theta}(\mathbf{z}))$$

Variational Autoencoders (VAEs)

How to maximize the ELBO?

ELBO

$$\mathcal{L}(\theta, \phi; \mathbf{x}^{(i)}) = -D_{KL}(q_{\phi}(\mathbf{z}|\mathbf{x}^{(i)})||p_{\theta}(\mathbf{z})) + \mathbb{E}_{q_{\phi}(\mathbf{z}|\mathbf{x}^{(i)})} [\log p_{\theta}(\mathbf{x}^{(i)}|\mathbf{z})] \quad (3)$$

We want to differentiate and optimize the lower bound $\mathcal{L}(\theta, \phi; \mathbf{x}^{(i)})$ w.r.t. both the variational parameters ϕ and generative parameters θ . However, the gradient of the lower bound w.r.t. ϕ is a bit problematic. The usual (naïve) Monte Carlo gradient estimator for this type of problem is: $\nabla_{\phi} \mathbb{E}_{q_{\phi}(\mathbf{z})} [f(\mathbf{z})] = \mathbb{E}_{q_{\phi}(\mathbf{z})} [f(\mathbf{z}) \nabla_{q_{\phi}(\mathbf{z})} \log q_{\phi}(\mathbf{z})] \simeq \frac{1}{L} \sum_{l=1}^L f(\mathbf{z}^{(l)}) \nabla_{q_{\phi}(\mathbf{z}^{(l)})} \log q_{\phi}(\mathbf{z}^{(l)})$ where $\mathbf{z}^{(l)} \sim q_{\phi}(\mathbf{z}|\mathbf{x}^{(i)})$. This gradient estimator exhibits very high variance (see e.g. [BJP12]) and is impractical for our purposes.

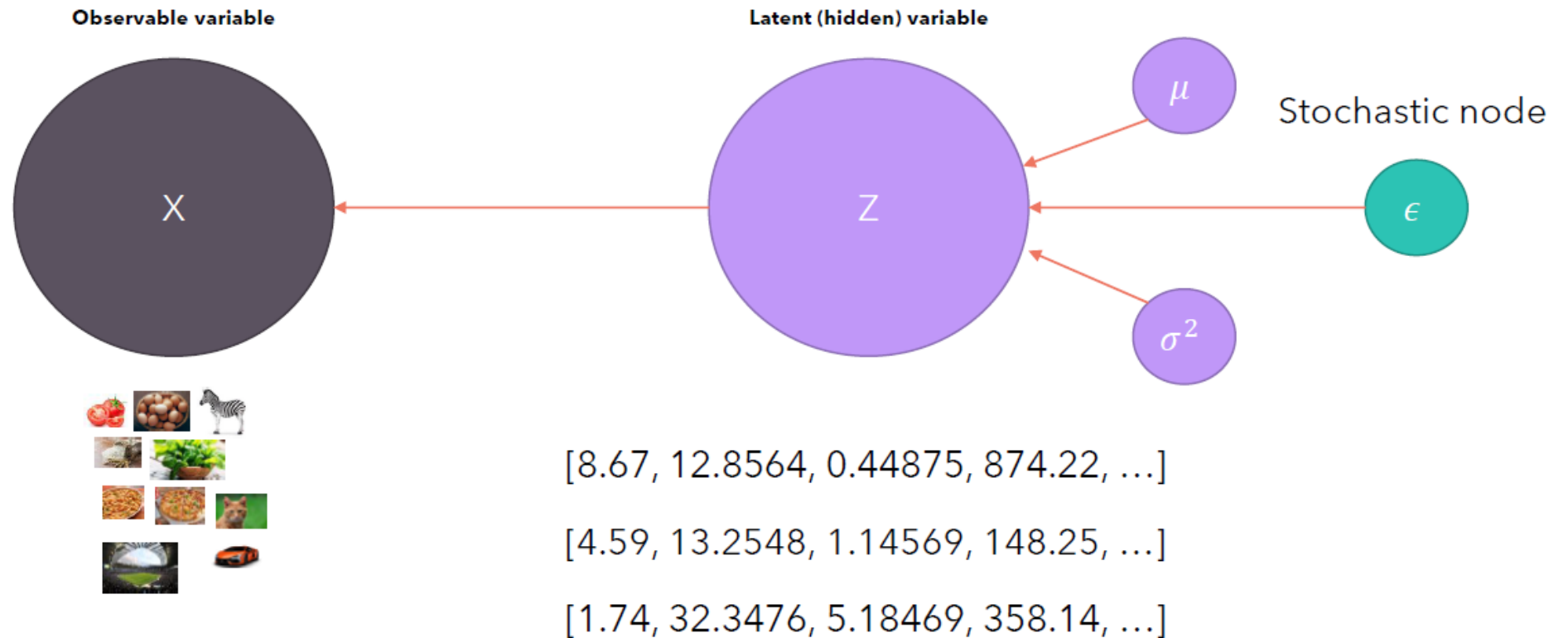
SCORE estimator

Kingma, D.P. and Welling, M., 2013. Auto-encoding variational bayes. *arXiv preprint arXiv:1312.6114*.

This estimator is **unbiased**, meaning that even if at every step it may not be equal to the true expectation, on average it will converge to it, but as it is stochastic, it also has a variance and it happens to be high for practical use. Plus, we can't run backpropagation through it!

Variational Autoencoders (VAEs)

The reparameterization trick



Variational Autoencoders (VAEs)

Running backpropagation on the reparametrized model

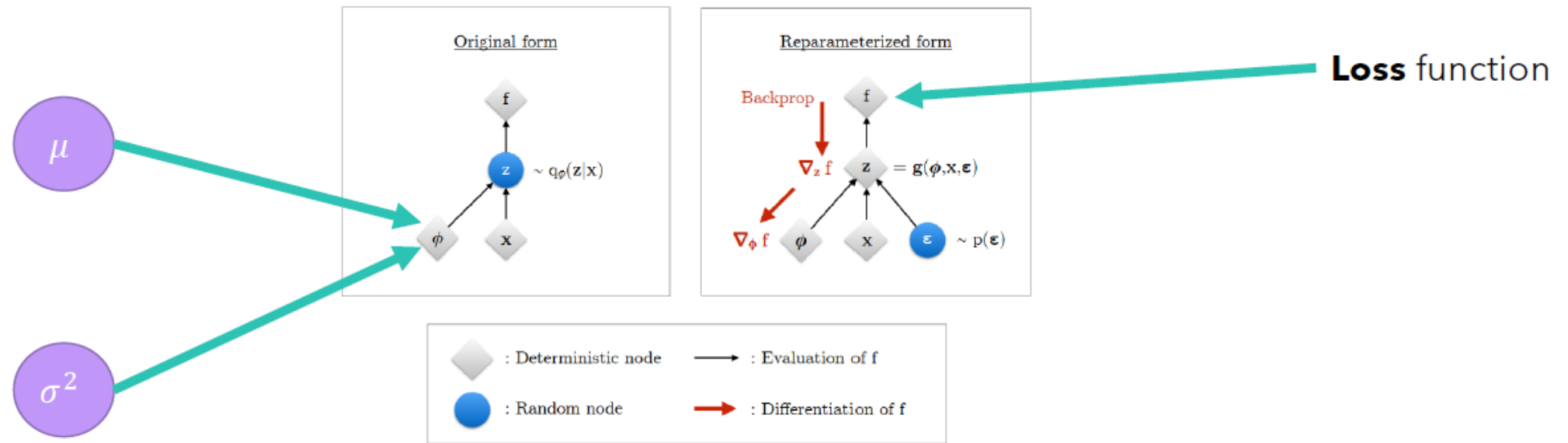


Figure 2.3: Illustration of the reparameterization trick. The variational parameters ϕ affect the objective f through the random variable $z \sim q_\phi(z|x)$. We wish to compute gradients $\nabla_\phi f$ to optimize the objective with SGD. In the original form (left), we cannot differentiate f w.r.t. ϕ , because we cannot directly backpropagate gradients through the random variable z . We can 'externalize' the randomness in z by re-parameterizing the variable as a deterministic and differentiable function of ϕ , x , and a newly introduced random variable ϵ . This allows us to 'backprop through z ', and compute gradients $\nabla_\phi f$.

Kingma, D.P. and Welling, M., 2019. An introduction to variational autoencoders. *Foundations and Trends® in Machine Learning*, 12(4), pp.307-392.

Variational Autoencoders (VAEs)

A new estimator!

$$L(\theta, \phi, x) = E_{q_\phi(z|x)} \left[\underbrace{\log \frac{p_\theta(x, z)}{q_\phi(z|x)}}_{\text{ELBO}} \right] = E_{p(\epsilon)} \left[\log \frac{p_\theta(x, z)}{q_\phi(z|x)} \right]$$

$$\epsilon \approx p(\epsilon)$$

ELBO

$$z = g(\phi, x, \epsilon)$$

$$E_{p(\epsilon)} \left[\log \frac{p_\theta(x, z)}{q_\phi(z|x)} \right] \cong \tilde{\mathcal{L}}(\theta, \phi, x) = \log \frac{p_\theta(x, z)}{q_\phi(z|x)}$$

aging the approximate posterior to be close to the prior $p_\theta(z)$. This yields a second version of the SGVB estimator $\tilde{\mathcal{L}}^B(\theta, \phi; \mathbf{x}^{(i)}) \simeq \mathcal{L}(\theta, \phi; \mathbf{x}^{(i)})$, corresponding to eq. (3), which typically has less variance than the generic estimator:

Kingma, D.P. and Welling, M., 2013. Auto-encoding variational bayes. *arXiv preprint arXiv:1312.6114*.

Variational Autoencoders (VAEs)

Is the new estimator unbiased?

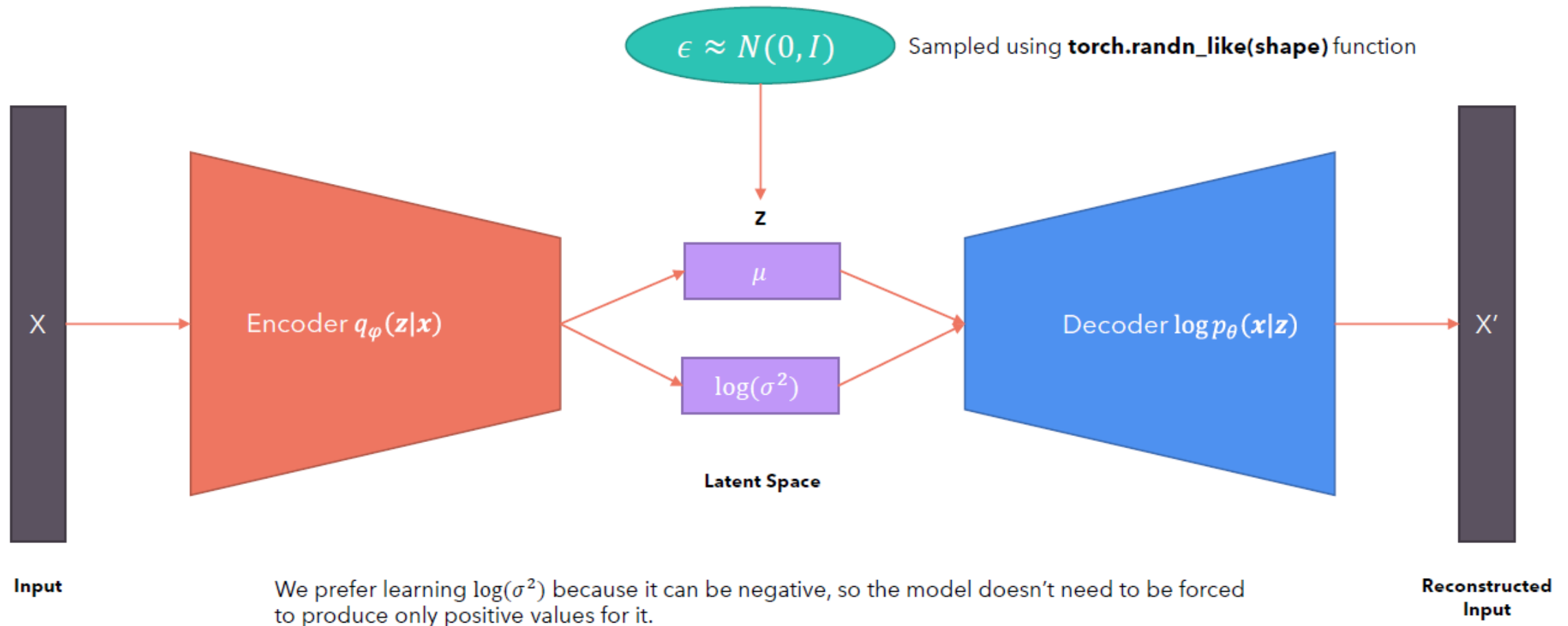
$$L(\theta, \varphi, x) = E_{p(\epsilon)} \left[\log \frac{p_{\theta}(x, z)}{q_{\varphi}(z|x)} \right] \quad \tilde{L}(\theta, \varphi, x) = \log \frac{p_{\theta}(x, z)}{q_{\varphi}(z|x)}$$

$$\begin{aligned} E_{p(\epsilon)} [\nabla_{\theta, \varphi} \tilde{L}(\theta, \varphi, x)] &= E_{p(\epsilon)} \left[\nabla_{\theta, \varphi} \log \frac{p_{\theta}(x, z)}{q_{\varphi}(z|x)} \right] \\ &= \nabla_{\theta, \varphi} (E_{p(\epsilon)} \left[\log \frac{p_{\theta}(x, z)}{q_{\varphi}(z|x)} \right]) \\ &= \nabla_{\theta, \varphi} (L(\theta, \varphi, x)) \end{aligned}$$



Variational Autoencoders (VAEs)

Example network



Variational Autoencoders (VAEs)

Show me the loss already!

MLP = Multi Layer Perceptron

Let the prior over the latent variables be the centered isotropic multivariate Gaussian $p_{\theta}(\mathbf{z}) = \mathcal{N}(\mathbf{z}; \mathbf{0}, \mathbf{I})$. Note that in this case, the prior lacks parameters. We let $p_{\theta}(\mathbf{x}|\mathbf{z})$ be a multivariate Gaussian (in case of real-valued data) or Bernoulli (in case of binary data) whose distribution parameters are computed from $\mathbf{z}$ with a MLP (a fully-connected neural network with a single hidden layer, see appendix C). Note the true posterior $p_{\theta}(\mathbf{z}|\mathbf{x})$ is in this case intractable. While there is much freedom in the form $q_{\phi}(\mathbf{z}|\mathbf{x})$, we'll assume the true (but intractable) posterior takes on an approximate Gaussian form with an approximately diagonal covariance. In this case, we can let the variational approximate posterior be a multivariate Gaussian with a diagonal covariance structure:

$$\log q_{\phi}(\mathbf{z}|\mathbf{x}^{(i)}) = \log \mathcal{N}(\mathbf{z}; \boldsymbol{\mu}^{(i)}, \boldsymbol{\sigma}^{2(i)} \mathbf{I}) \quad (9)$$

where the mean and s.d. of the approximate posterior, $\boldsymbol{\mu}^{(i)}$ and $\boldsymbol{\sigma}^{(i)}$, are outputs of the encoding MLP, i.e. nonlinear functions of datapoint $\mathbf{x}^{(i)}$ and the variational parameters ϕ (see appendix C).

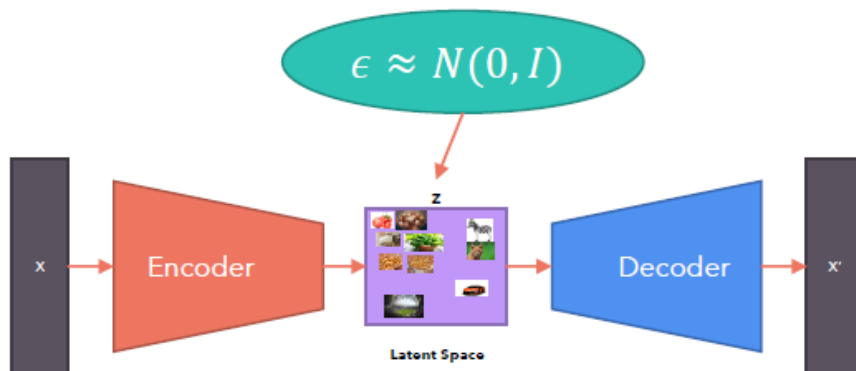
As explained in section 2.4, we sample from the posterior $\mathbf{z}^{(i,l)} \sim q_{\phi}(\mathbf{z}|\mathbf{x}^{(i)})$ using $\mathbf{z}^{(i,l)} = g_{\phi}(\mathbf{x}^{(i)}, \boldsymbol{\epsilon}^{(l)}) = \boldsymbol{\mu}^{(i)} + \boldsymbol{\sigma}^{(i)} \odot \boldsymbol{\epsilon}^{(l)}$ where $\boldsymbol{\epsilon}^{(l)} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$. With $\odot$ we signify an element-wise product. In this model both $p_{\theta}(\mathbf{z})$ (the prior) and $q_{\phi}(\mathbf{z}|\mathbf{x})$ are Gaussian; in this case, we can use the estimator of eq. (7) where the KL divergence can be computed and differentiated without estimation (see appendix B). The resulting estimator for this model and datapoint $\mathbf{x}^{(i)}$ is:

$$\mathcal{L}(\theta, \phi; \mathbf{x}^{(i)}) \simeq \frac{1}{2} \sum_{j=1}^J \left(1 + \log((\sigma_j^{(i)})^2) - (\mu_j^{(i)})^2 - (\sigma_j^{(i)})^2 \right) + \frac{1}{L} \sum_{l=1}^L \log p_{\theta}(\mathbf{x}^{(i)}|\mathbf{z}^{(i,l)}) \quad (10)$$

where $\mathbf{z}^{(i,l)} = \boldsymbol{\mu}^{(i)} + \boldsymbol{\sigma}^{(i)} \odot \boldsymbol{\epsilon}^{(l)}$ and $\boldsymbol{\epsilon}^{(l)} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$

As explained above and in appendix C, the decoding term $\log p_{\theta}(\mathbf{x}^{(i)}|\mathbf{z}^{(i,l)})$ is a Bernoulli or Gaussian MLP, depending on the type of data we are modelling.

Kingma, D.P. and Welling, M., 2013. Auto-encoding variational bayes. *arXiv preprint arXiv:1312.6114*.



Variational Autoencoders (VAEs)

How to derive the loss function?



The encoder distribution is $q(z|x) = \mathcal{N}(z|\mu(x), \Sigma(x))$ where $\Sigma = \text{diag}(\sigma_1^2, \dots, \sigma_n^2)$.

48

The latent prior is given by $p(z) = \mathcal{N}(0, I)$.



Both are multivariate Gaussians of dimension n , for which in general the KL divergence is:



$$\mathcal{D}_{\text{KL}}[p_1 || p_2] = \frac{1}{2} \left[\log \frac{|\Sigma_2|}{|\Sigma_1|} - n + \text{tr}\{\Sigma_2^{-1}\Sigma_1\} + (\mu_2 - \mu_1)^T \Sigma_2^{-1} (\mu_2 - \mu_1) \right]$$



where $p_1 = \mathcal{N}(\mu_1, \Sigma_1)$ and $p_2 = \mathcal{N}(\mu_2, \Sigma_2)$.



In the VAE case, $p_1 = q(z|x)$ and $p_2 = p(z)$, so $\mu_1 = \mu$, $\Sigma_1 = \Sigma$, $\mu_2 = \vec{0}$, $\Sigma_2 = I$. Thus:

$$\begin{aligned} \mathcal{D}_{\text{KL}}[q(z|x) || p(z)] &= \frac{1}{2} \left[\log \frac{|\Sigma_2|}{|\Sigma_1|} - n + \text{tr}\{\Sigma_2^{-1}\Sigma_1\} + (\mu_2 - \mu_1)^T \Sigma_2^{-1} (\mu_2 - \mu_1) \right] \\ &= \frac{1}{2} \left[\log \frac{|I|}{|\Sigma|} - n + \text{tr}\{I^{-1}\Sigma\} + (\vec{0} - \mu)^T I^{-1} (\vec{0} - \mu) \right] \\ &= \frac{1}{2} \left[-\log |\Sigma| - n + \text{tr}\{\Sigma\} + \mu^T \mu \right] \\ &= \frac{1}{2} \left[-\log \prod_i \sigma_i^2 - n + \sum_i \sigma_i^2 + \sum_i \mu_i^2 \right] \\ &= \frac{1}{2} \left[-\sum_i \log \sigma_i^2 - n + \sum_i \sigma_i^2 + \sum_i \mu_i^2 \right] \\ &= \frac{1}{2} \left[-\sum_i (\log \sigma_i^2 + 1) + \sum_i \sigma_i^2 + \sum_i \mu_i^2 \right] \end{aligned}$$



Variational Autoencoders (VAEs) – Explore!

- Practice math behind the concept along with the derivations thought in class..!
- Realtime Applications of VAE – Particularly Image Generation and Data Compression
- Strength of VAE
- Weakness of VAE
- Difference between GAN and VAE
- Difference between AE and VAE

Dept. of Machine Learning, BMSCE

Thank you!!!

Dept. of Machine Learning, BMSCE